

UNIVERSIDAD DE LOS LLANOS
Facultad de Ciencias Básicas e Ingeniería
Programa de Ingeniería de Sistemas

Simulación de eventos discretos

Línea de espera G/G/1, red de colas, inventario probabilístico,
SimPy y AnyLogic

Informe del Laboratorio 4
(guía FO-DOC-112, Práctica N.º 03)
Curso: Simulación Computacional

Presentado por:
Duván Felipe Baquero
Código: 160005002

Docente:
Ángel Alfonso Cruz Roa

Villavicencio, Meta
1 de octubre de 2026

Índice

1	Objetivos	2
2	Consulta previa	2
2.1	¿En qué consiste el paradigma de simulación de eventos discretos?	2
2.2	¿Cuáles son los elementos y la metodología de modelado de la simulación de eventos discretos?	2
2.3	¿En qué consiste un sistema de cola simple?	3
2.4	¿En qué consiste una red de colas?	3
2.5	¿En qué consiste un inventario probabilístico?	3
2.6	¿Qué es el framework SimPy para simulación de eventos discretos?	3
2.7	¿Cómo usar el software de simulación AnyLogic para simulación de eventos discretos?	4
3	Metodología y herramientas	4
4	Numeral 5.1: sistema de línea de espera con un servidor	4
4.1	El modelo y los cambios hechos al ejemplo	4
4.2	Escenario A	5
4.3	Escenario B	6
4.4	Diez simulaciones por escenario	7
4.5	¿Qué puede analizar de los resultados comparando los dos escenarios?	9
5	Numeral 5.2: red de colas	9
5.1	El modelo	9
5.2	Correcciones al cuaderno de ejemplo	10
5.3	Escenario A	11
5.4	Escenario B	12
5.5	Veinte repeticiones por escenario	13
5.6	¿Qué puede decir de la media y la desviación estándar de la muestra para cada medida?	15
5.7	¿Qué análisis puede realizar de los intervalos de confianza del 95 %?	15
6	Numeral 5.3: inventario probabilístico	16
6.1	El modelo y sus parámetros	16
6.2	Correcciones al pseudocódigo del libro	17
6.3	Resultados de la simulación principal	17
6.4	Réplicas y efecto de la tasa de llegada	19
6.5	Análisis	20
7	Numeral 5.4: simulación de eventos discretos en SimPy	21
7.1	Instalación	21
7.2	Tutorial de Stefan Scherfke	21
7.3	Tutorial <i>SimPy in 10 minutes</i>	22
7.4	Ejemplos de SimPy	22
8	Numeral 5.5: simulación de eventos discretos en AnyLogic	23
8.1	Bank office model (sistema de colas)	24
8.2	Subway entrance (peatones)	26
8.3	Intersection (tráfico vehicular)	29
8.4	Job shop (modelado de procesos)	32
8.5	Lead acid battery production (manejo de materiales)	33
9	Conclusiones	37
10	Anexos	38

1. Objetivos

Estos son los objetivos que trae la guía de laboratorio:

- Introducir al paradigma de simulación de eventos discretos por medio de sus elementos, diseño, metodología de modelado e implementación.
- Apropiar la metodología de modelado e implementación de simulación de eventos discretos a partir de dos casos de uso: un sistema de línea de espera con un servidor y una red de colas.
- Implementar la metodología de modelado y simulación de eventos discretos en un caso de estudio de un modelo de inventario probabilístico.
- Implementar casos de estudio de simulación de eventos discretos en frameworks o softwares de simulación para eventos discretos como SimPy y AnyLogic.

2. Consulta previa

2.1. ¿En qué consiste el paradigma de simulación de eventos discretos?

En modelar un sistema cuyo estado solo cambia en instantes puntuales, llamados eventos o sucesos, y suponer que entre dos eventos seguidos no pasa nada relevante. En una fila de banco, por ejemplo, el número de clientes solo cambia cuando alguien llega o cuando alguien termina de ser atendido. Por eso el reloj de la simulación no avanza de a poco, sino que salta de un evento al siguiente (simulación asíncrona). En cada salto se actualizan el estado y los contadores y se programan los eventos futuros que el evento actual genera. Esto lo diferencia de la simulación continua, donde el estado cambia todo el tiempo y se describe con ecuaciones diferenciales, y de la simulación síncrona, que avanza en pasos fijos Δt aunque no ocurra nada.

2.2. ¿Cuáles son los elementos y la metodología de modelado de la simulación de eventos discretos?

Según [1], los elementos básicos son las variables y los sucesos. Las variables son de tres tipos:

- la **variable de tiempo** t , que lleva el tiempo simulado transcurrido;
- las **variables de estado**, que describen el sistema en el instante t (número de clientes en cola, nivel del inventario);
- los **contadores**, que acumulan lo que ha pasado hasta t (llegadas, salidas, costos) y con los que al final se calculan las medidas de desempeño.

A esto se suma una **lista de sucesos**, que en los ejemplos del curso es la estructura TSuc: guarda el instante de cada suceso pendiente y un valor muy grande M cuando no hay ninguno. El programa principal repite siempre lo mismo: busca en la lista el suceso más próximo, avanza el reloj hasta él y ejecuta la rutina de ese tipo de suceso, que cambia el estado, actualiza contadores y programa sucesos nuevos. Cuando la lista queda vacía se calculan las salidas.

La metodología de modelado es iterativa: se define el problema y las fronteras del sistema, se formulan hipótesis, se construye un modelo básico y luego uno simplificado que sea tratable, se

implementa, se verifica (que el programa haga lo que el modelo dice), se valida (que el modelo represente el sistema real) y se experimenta con él. Si una etapa no es satisfactoria se vuelve a la anterior. Como la salida es aleatoria, la experimentación exige hacer varias réplicas y resumirlas con medias, desviaciones e intervalos de confianza.

2.3. ¿En qué consiste un sistema de cola simple?

En un sistema con una sola fuente de clientes, una sola cola y un solo servidor. Los clientes llegan según un proceso aleatorio; si el servidor está libre se atienden de inmediato y si no, esperan en la cola, casi siempre en orden de llegada (FIFO). Se describe con la notación de Kendall: G/G/1 significa tiempos entre llegadas y de servicio con distribuciones generales y un servidor, y M/M/1 el caso particular en que ambos son exponenciales. El parámetro que más pesa es la intensidad de tráfico $\rho = \lambda/\mu$, el cociente entre la tasa de llegadas y la de servicio. Si $\rho < 1$ el sistema alcanza un régimen estable, y si $\rho \geq 1$ la cola crece sin límite. Las medidas usuales son el tiempo medio en el sistema y en la cola, el número medio de clientes y la utilización del servidor.

2.4. ¿En qué consiste una red de colas?

En un conjunto de nodos de servicio, cada uno con su cola, conectados de forma que la salida de uno es la entrada de otro. Un cliente recorre varios nodos según reglas de ruteo, que pueden ser fijas o probabilísticas. En la red del numeral 5.2, por ejemplo, el 40 % de los vehículos pasa de la ventanilla a una nave auxiliar y el 60 % va directo a la nave principal. Las redes pueden ser abiertas (los clientes entran y salen, como en ese caso) o cerradas (circula siempre la misma población). Algunos casos tienen solución analítica, como las redes de Jackson con servicios exponenciales, pero en cuanto aparecen servicios normales o tiempos que dependen del estado de la cola, como en la ITV, la simulación es la única herramienta práctica.

2.5. ¿En qué consiste un inventario probabilístico?

En un sistema de almacenamiento en el que la demanda, los tiempos de entrega o ambos son aleatorios. La empresa tiene que decidir cuándo pedir y cuánto pedir (su política de inventario) para equilibrar tres costos que se contraponen: el de almacenar mucho, el de pedir seguido y el de quedarse sin producto y perder ventas. En el modelo de [1] los clientes llegan como un proceso de Poisson, cada uno pide una cantidad aleatoria y la política es de tipo (p, P) : cuando el inventario baja de p y no hay pedido pendiente, se pide lo necesario para volver a P , y el pedido tarda L en llegar. Como no hay fórmulas cerradas para este caso, el beneficio, la fracción de clientes satisfechos y el tiempo con inventario en cero se estiman simulando.

2.6. ¿Qué es el framework SimPy para simulación de eventos discretos?

SimPy es una librería de Python, de código abierto, para simulación de eventos discretos basada en procesos. Los procesos (clientes, máquinas, vehículos) se escriben como funciones generadoras de Python, y cada `yield` de un evento suspende el proceso hasta que ese evento ocurra. Un `Environment` maneja el reloj y la cola de eventos, y hay recursos compartidos ya hechos: `Resource` (servidores con cola), `Container` (cantidades continuas o a granel) y `Store` (objetos individuales), con variantes con prioridad y con desplazamiento. La diferencia con la implementación de los nu-

merales 5.1 a 5.3 es que en SimPy no se programa la lista de sucesos: se describe lo que hace cada entidad y la librería decide en qué orden se ejecuta todo.

2.7. ¿Cómo usar el software de simulación AnyLogic para simulación de eventos discretos?

AnyLogic es un entorno gráfico de simulación que combina eventos discretos, sistemas dinámicos y modelado basado en agentes. Para eventos discretos se usa sobre todo la *Process Modeling Library*: el modelo se arma como un diagrama de flujo con bloques que se arrastran desde la paleta y se conectan (**Source** crea las entidades, **Queue** las pone en cola, **Delay** y **Service** las retienen un tiempo, **SelectOutput** las enruta, **ResourcePool** define recursos y **Sink** las elimina). Cada bloque se configura en su panel de propiedades con distribuciones como `triangular()` o `exponential()`. Encima del diagrama se dibuja el espacio (nodos, rutas, bandas transportadoras) y se añade animación 2D o 3D. Para casos especializados están la *Pedestrian Library*, la *Road Traffic Library*, la *Material Handling Library* y la *Fluid Library*. Al ejecutar, los bloques recogen estadísticas por su cuenta (utilización, tamaño de cola) que se pueden llevar a gráficas. La versión gratuita PLE (*Personal Learning Edition*) es la que se usó en el numeral 5.5.

3. Metodología y herramientas

Los numerales 5.1 a 5.4 se desarrollaron en cuatro cuadernos de Jupyter en Python, ejecutados de principio a fin y adjuntos a este informe:

Numeral	Archivo
5.1 Línea de espera G/G/1	Lab4_5.1_GG1_160005002.ipynb
5.2 Red de colas	Lab4_5.2_RedColas_160005002.ipynb
5.3 Inventario probabilístico	Lab4_5.3_Inventario_160005002.ipynb
5.4 SimPy	Lab4_5.4_SimPy_160005002.ipynb

Se usaron `numpy`, `pandas`, `matplotlib`, `scipy` y `simpy` 4.1.2, y los cuadernos corren en Google Colab sin instalar nada más que SimPy. Todas las semillas están fijas, así que cada ejecución reproduce exactamente las tablas y figuras de este informe. El numeral 5.5 se desarrolló en AnyLogic 8.9 PLE; los modelos `.alp` se entregan en la carpeta `Modelos_AnyLogic`.

Los numerales 5.1 y 5.2 parten de copias de los cuadernos de ejemplo del curso y conservan su estructura: variables globales, una rutina por tipo de suceso y un bucle principal que recorre la lista `TSuc`. Al correrlos aparecieron dos errores de cálculo en los ejemplos, uno en cada cuaderno, que se corrigieron y se documentan en su sección. El numeral 5.3 se escribió desde cero siguiendo el pseudocódigo de la sección 5.6 de [1], con la misma estructura de los otros dos.

4. Numeral 5.1: sistema de línea de espera con un servidor

4.1. El modelo y los cambios hechos al ejemplo

El cuaderno de ejemplo implementa la cola G/G/1 de la sección 5.5.1 de [1]. Hay dos sucesos, la llegada de un cliente (`tLL`) y la salida del servidor (`tS`). El estado es el número de clientes n

en el sistema. Se aceptan llegadas hasta el instante T y después el servidor sigue trabajando hasta vaciar la cola. Los números uniformes salen de dos generadores congruenciales mixtos, uno para las llegadas y otro para los servicios.

Sobre ese código se hicieron cuatro cambios:

1. **GenerarX** y **GenerarY** toman los parámetros del escenario en lugar de tener `vlambda = 5` fijo. Se conservó la convención del ejemplo, en la que λ es una tasa: $X = -\ln(1 - U)/\lambda$, con media $1/\lambda$.
2. Para el escenario B se agregó un generador normal por el método polar, que toma sus uniformes de `gencongru2`, y el truncamiento en cero que pide la nota de la guía. Antes de usarlo se probaron 20 000 valores: media 0,013, desviación 1,002 y prueba de Kolmogorov–Smirnov con $p = 0,17$.
3. El bucle principal quedó dentro de una función `simular()` para poder repetirlo, y los generadores se reinician al comenzar cada escenario. Así la corrida 1 es reproducible y las corridas 2 a 10 siguen la secuencia.
4. **Corrección de los tiempos medios.** Las listas `LL`, `S` y `Serv` empiezan con un valor ficticio 0 en la posición 0, de modo que el cliente i queda en la posición i . El ejemplo sumaba las posiciones $0, \dots, \text{NLL} - 1$, así que incluía el cero ficticio y dejaba por fuera al último cliente. Ahora se suman las posiciones $1, \dots, \text{NLL}$. En la primera corrida del escenario A el tiempo medio en el sistema pasa de 13,0940 con el cálculo original a 13,2105 con el corregido.

4.2. Escenario A

$T = 40$, tiempos entre llegadas exponenciales con $\lambda = 5$ y tiempos de servicio exponenciales con $\lambda = 3$. Llegan 5 clientes por unidad de tiempo y el servidor puede atender 3, así que $\rho = 5/3 \approx 1,67$: el sistema es inestable.

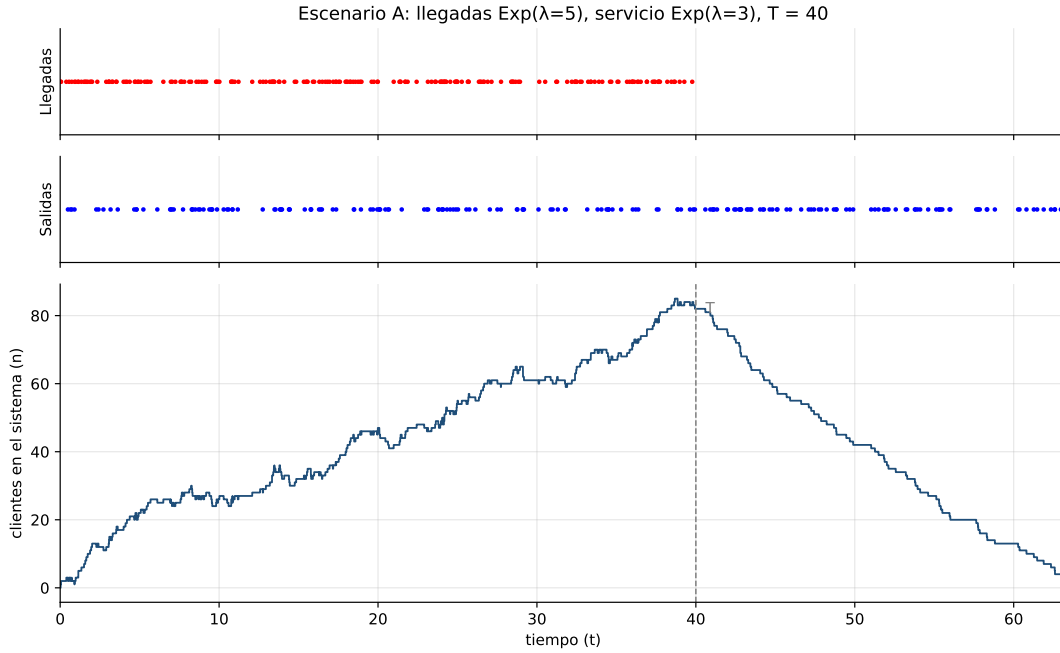


Figura 1: Escenario A, primera corrida: instantes de llegada, instantes de salida y número de clientes en el sistema. La línea punteada marca $T = 40$.

Respuestas con la primera corrida:

- Tiempo medio de los clientes en el sistema: 13,2105.
- Tiempo medio de los clientes en la cola: 12,8973.
- Tiempo transcurrido desde T hasta que el último cliente abandona el sistema: 23,3130.
- Número máximo de clientes en el sistema: 85.
- Total de clientes que pasaron por el sistema: 202.

La figura 1 muestra lo que se espera con $\rho > 1$: el número de clientes sube casi en línea recta durante toda la ventana de llegadas, llega a 85 justo en T y a partir de ahí solo baja. La diferencia entre el tiempo en el sistema y el tiempo en cola es 0,31, muy cerca del tiempo medio de servicio de $1/3$. Un cliente pasa casi todo su tiempo esperando.

4.3. Escenario B

$T = 40$, tiempos entre llegadas exponenciales con $\lambda = 1$ y tiempos de servicio normales con $\mu = 2$ y $\sigma = 1$, truncados en cero. Ahora $\rho \approx 2$. El truncamiento casi no cambia la media, porque $P\{Y < 0\} = \Phi(-2) \approx 2,3\%$.

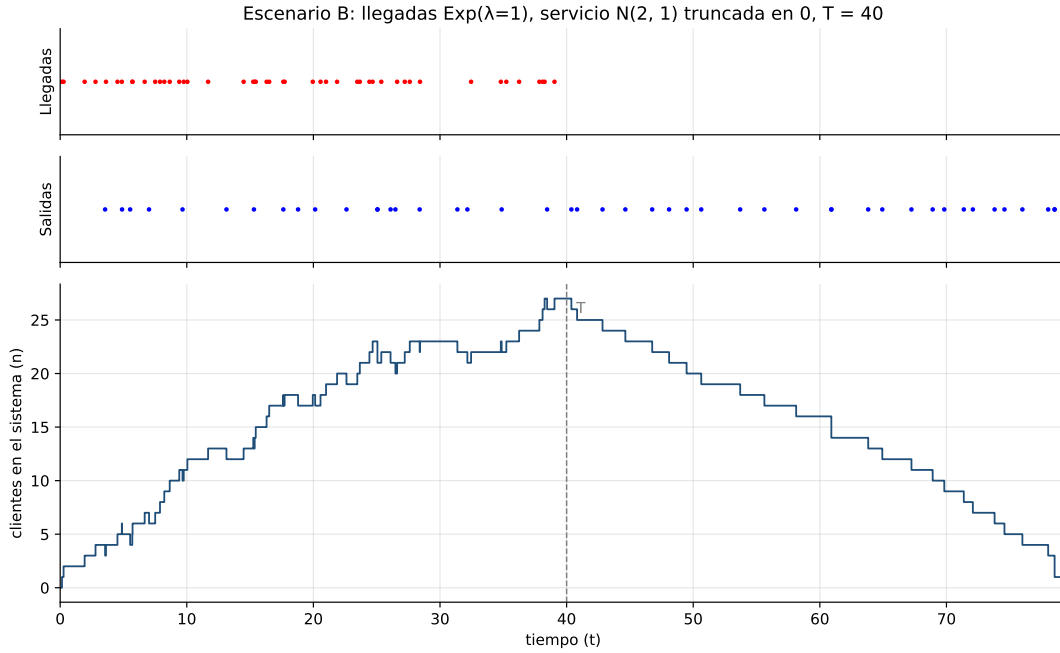


Figura 2: Escenario B, primera corrida.

Respuestas con la primera corrida:

- Tiempo medio de los clientes en el sistema: 26,1985.
- Tiempo medio de los clientes en la cola: 24,5107.
- Tiempo transcurrido desde T hasta que el último cliente abandona el sistema: 39,4699.
- Número máximo de clientes en el sistema: 27.
- Total de clientes que pasaron por el sistema: 47.

En esta corrida entraron 47 clientes, más de los 40 esperados, y la media de los servicios fue 1,69 en lugar de 2; 3 de los 47 quedaron truncados en cero. Con 47 datos y $\sigma \approx 1$, una media de 1,69 está a unos dos errores estándar de 2, así que es variación de la muestra y no un defecto del generador, como ya había confirmado la prueba de los 20 000 valores.

4.4. Diez simulaciones por escenario

Las tablas 1 y 2 tienen las diez corridas de cada escenario y la tabla 3 las resume en el formato que pide la guía.

Tabla 1: Escenario A: diez corridas.

Corrida	T. medio sistema	T. medio cola	Desde T hasta el último	Máx. clientes	Total clientes
1	13,2105	12,8973	23,3130	85	202
2	12,0779	11,7775	22,5801	77	208
3	13,0522	12,7117	22,6776	76	183
4	6,1492	5,8583	10,3846	42	163
5	13,9198	13,5397	27,9925	69	177
6	14,0110	13,6833	23,2034	80	190
7	19,4143	19,0511	41,4152	109	224
8	11,7507	11,4337	25,2216	74	204
9	14,8242	14,4811	25,8949	82	191
10	11,3318	11,0000	25,2062	82	195

Tabla 2: Escenario B: diez corridas.

Corrida	T. medio sistema	T. medio cola	Desde T hasta el último	Máx. clientes	Total clientes
1	26,1985	24,5107	39,4699	27	47
2	19,2413	17,3586	31,4515	20	37
3	23,1418	21,2682	43,3908	24	44
4	18,5432	16,5342	29,8294	16	34
5	22,2966	20,3283	43,1280	21	42
6	18,7623	16,7262	33,4140	18	36
7	21,6338	19,7010	41,3751	23	42
8	47,9425	45,5503	90,2462	38	54
9	11,0512	8,9061	13,3161	10	24
10	31,2140	29,0635	61,2186	30	47

Tabla 3: Medidas de desempeño para los escenarios A y B luego de 10 simulaciones (promedio $\pm$ desviación estándar muestral).

Medida de desempeño	Escenario A	Escenario B
Tiempo medio de los clientes en el sistema	12,9742 $\pm$ 3,3083	24,0025 $\pm$ 9,9269
Tiempo medio de los clientes en la cola	12,6434 $\pm$ 3,2887	21,9947 $\pm$ 9,8325
Tiempo transcurrido desde T hasta que el último cliente abandona el sistema	24,7889 $\pm$ 7,5262	42,6839 $\pm$ 20,7049
Número máximo de clientes en el sistema	77,6000 $\pm$ 16,4735	22,7000 $\pm$ 7,7896
Total de clientes que pasaron por el sistema	193,7000 $\pm$ 17,1791	40,7000 $\pm$ 8,3673

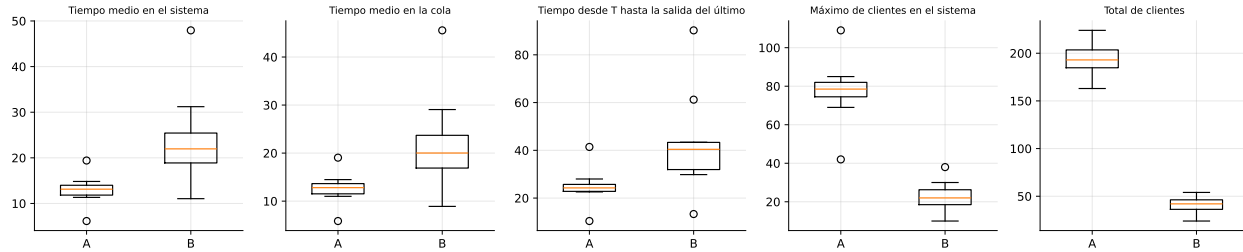


Figura 3: Diagramas de caja de las cinco medidas en las diez corridas de cada escenario.

4.5. ¿Qué puede analizar de los resultados comparando los dos escenarios?

Lo primero es que **los dos escenarios son inestables**: en A llegan 5 clientes por unidad de tiempo y se atienden 3 ($\rho = 1,67$), y en B llega 1 y se atiende 0,5 ($\rho = 2$). No hay un régimen estable. La cola crece mientras haya llegadas, y por eso todas las medidas dependen de T .

Un cálculo sencillo de “flujo” reproduce bastante bien la tabla 3. Si la cola crece a razón de $\lambda - \mu$ clientes por unidad de tiempo, al llegar a T hay unos $(\lambda - \mu)T$ clientes en el sistema. En A eso da $(5 - 3) \cdot 40 = 80$, frente a un máximo promedio de 77,6; vaciarlos a 3 por unidad de tiempo toma $80/3 \approx 26,7$, y el tiempo después de T salió 24,8. En B da $(1 - 0,5) \cdot 40 = 20$ frente a 22,7, y vaciarlos toma $20 \cdot 2 = 40$, frente a 42,7. Con el mismo razonamiento, un cliente que llega en el instante s encuentra unos $(\lambda - \mu)s$ clientes delante y espera cerca de $(\rho - 1)s$; promediando sobre $[0, T]$ el tiempo en el sistema es del orden de $(\rho - 1)T/2$, es decir, 13,3 en A y 20 en B, frente a 12,97 y 24,00 simulados.

B es peor para el cliente aunque reciba cinco veces menos gente. En A pasaron 194 clientes en promedio y en B 41, pero en B cada cliente estuvo 24,0 unidades de tiempo en el sistema, casi el doble que en A (13,0), y el último salió 42,7 unidades después de T , contra 24,8 en A. Lo que manda no es cuánta gente llega, sino cuánto excede la demanda a la capacidad: en B el servidor necesitaría el doble del tiempo disponible, y en A 1,67 veces.

B también es más variable. El coeficiente de variación del tiempo en el sistema es 25 % en A (3,31/12,97) y 41 % en B (9,93/24,00). En la corrida 8 de B el último cliente salió 90 unidades después de T y en la corrida 9 solo 13. Con unos 40 clientes por corrida, una racha de llegadas o de servicios largos pesa mucho; con los 200 de A esos efectos tienden a compensarse.

Por último, en los dos escenarios **la cola es casi todo el tiempo en el sistema**: la diferencia entre ambas medidas es 0,33 en A y 2,01 en B, que son justo los tiempos medios de servicio. Más del 90 % del tiempo de cada cliente se va en esperar. Mejorar un poco el servicio no alcanzaría; haría falta un segundo servidor o limitar las llegadas hasta dejar $\rho < 1$.

5. Numeral 5.2: red de colas

5.1. El modelo

La red de la sección 5.5.2 de [1] representa un centro de revisión técnico-mecánica (ITV) con tres nodos, todos con cola FIFO de capacidad infinita:

- **N1**, ventanilla de documentación y pago, con servicio $N(\mu_1, \sigma_1)$ truncado en 0;

- **N2**, nave auxiliar para las pruebas adicionales de los vehículos de más de 10 años (el 40%), con servicio exponencial de media μ_2 ;
- **N3**, nave principal por la que pasan todos, con servicio $N(\mu_{31}, \sigma_{31})$ si hay menos de 5 vehículos y $N(\mu_{32}, \sigma_{32})$ si hay 5 o más, porque los operarios se apuran cuando hay congestión.

Del nodo 1 el vehículo va a N2 con probabilidad 0,4 o directamente a N3 con probabilidad 0,6, y de N2 siempre pasa a N3. Se aceptan llegadas hasta $T = 300$ y se sigue atendiendo hasta que sale el último.

Convención de los parámetros. El ejemplo genera los tiempos entre llegadas con `np.random.exponential(L)` y los servicios de N2 con `np.random.exponential(mu2)`. NumPy interpreta ese argumento como la media (escala), no como la tasa, y se respetó esa convención: con $\lambda = 2$ llega en promedio un vehículo cada 2 unidades de tiempo. Leer λ como tasa daría un vehículo cada 0,5 unidades a una ventanilla que tarda 2,1 en promedio, y el nodo 1 recibiría cuatro veces más de lo que puede atender.

5.2. Correcciones al cuaderno de ejemplo

Al revisar los resultados del primer escenario apareció una inconsistencia: por N2 pasaron 59 vehículos con un servicio medio de 2,4, pero el número medio de vehículos en N2 salía 0,19. Por la ley de Little debía rondar 0,46. La causa es que en el ejemplo **cada rutina acumula el área $n_j(t_{suc} - t)$ solo del nodo que ella misma modifica**, pero el reloj t avanza con los sucesos de todos los nodos. Si entre dos sucesos del nodo 2 ocurre, por ejemplo, una salida del nodo 3, ese tramo se pierde para el nodo 2. El resultado es que `n_med_n1`, `n_med_n2` y `n_med_n3` salen subestimados. Se corrigió con una función `acumular_areas()` que, al comienzo de cada suceso, suma el área de los tres nodos antes de mover el reloj.

Había un segundo detalle: el bucle guardaba el estado (t, n_1, n_2, n_3) una vez por vuelta, aunque en una vuelta se pueden procesar hasta cuatro sucesos, y así se perdían estados intermedios y posibles máximos. Ahora el estado se guarda después de cada suceso. Ninguno de los dos cambios toca las llamadas al generador, así que con la misma semilla se simula exactamente la misma historia y solo cambia la forma de medir.

Tabla 4: Número medio de vehículos por nodo en la primera repetición del escenario A: cálculo corregido, verificación directa $\sum_i (S_j(i) - LL_j(i))/t$ y cálculo del ejemplo original.

Nodo	Corregido	$\sum(S - LL)/t$	Ejemplo original
N1	3,7653	3,7653	2,6697
N2	0,4598	0,4598	0,1936
N3	23,1522	23,1522	16,9373

La tabla 4 confirma la corrección. Cuando la cola es FIFO y todos los clientes salen antes del final, el área bajo la curva de $n_j(t)$ es exactamente la suma de los tiempos de permanencia en el nodo, y los dos cálculos coinciden en todos los decimales. El error del ejemplo no era pequeño: en N3 subestimaba en un 27 %.

5.3. Escenario A

$\lambda = 2$; N1: $N(2,1; 0,5)$; N2: exponencial con $\mu_2 = 2,4$; N3: $N(4,1; 2,5)$ si $n_3 < 5$ y $N(3,0; 2,0)$ si $n_3 \geq 5$.

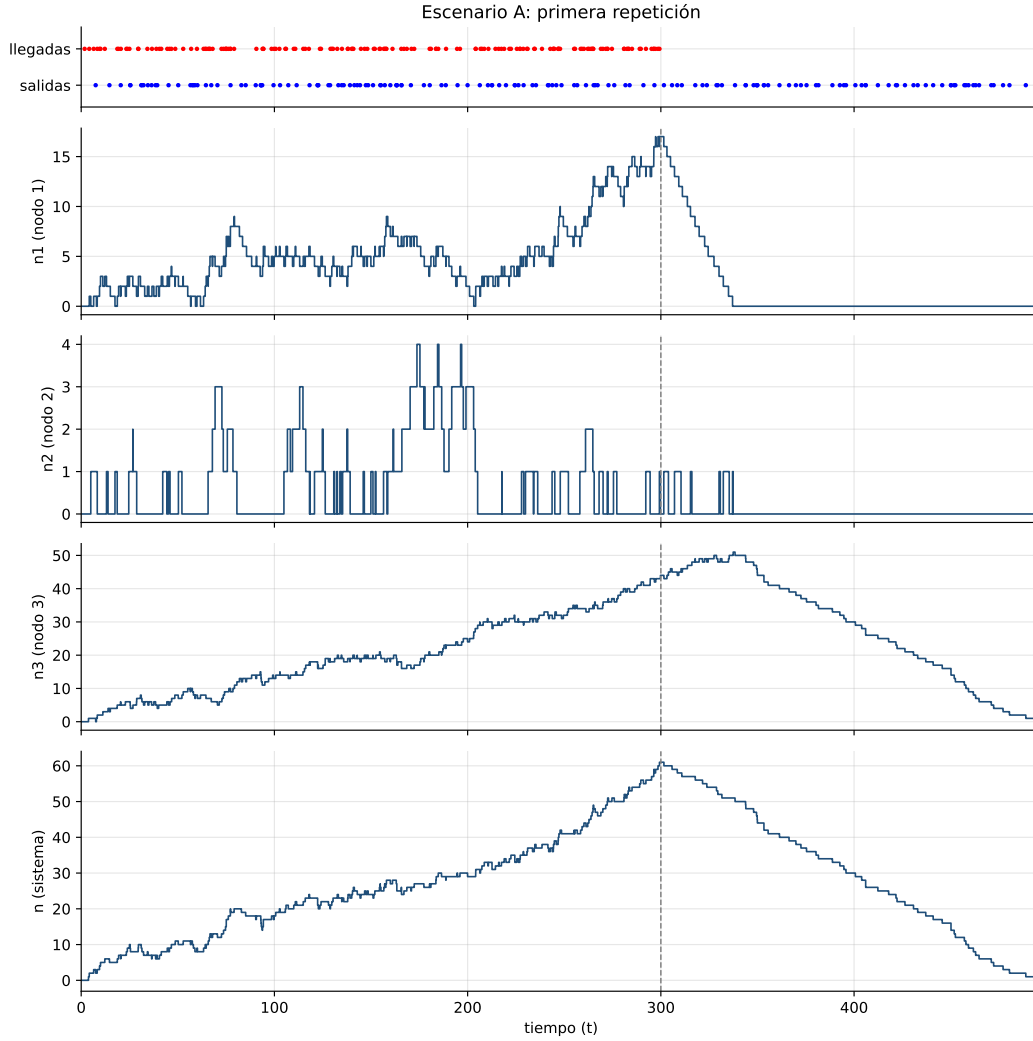


Figura 4: Escenario A, primera repetición: llegadas y salidas, número de vehículos en cada nodo y en el sistema.

Tabla 5: Respuestas a las preguntas con la primera repetición de cada escenario.

Medida	Escenario A	Escenario B
Tiempo medio en el sistema ($t_{med_sistema}$)	87,2638	23,2720
Número promedio en el nodo 1 (n_{med_n1})	3,7653	3,2319
Número promedio en el nodo 2 (n_{med_n2})	0,4598	0,1787
Número promedio en el nodo 3 (n_{med_n3})	23,1522	2,9987
Tiempo desde T hasta que sale el último	196,7626	26,7854
Número máximo de clientes en el sistema	61	15
Clientes que pasaron por el nodo 1 (n_1)	156	90
Clientes que pasaron por el nodo 2 (n_2)	59	36
Clientes que pasaron por el nodo 3 (n_3)	156	90
Total de clientes que pasaron por el sistema	156	90

En la figura 4 se ve dónde está el problema del escenario A. N1 se congestiona por ratos pero se recupera; N2 pasa el 57% del tiempo vacío; N3, en cambio, crece sin parar hasta T . A N3 llegan unos 0,48 vehículos por unidad de tiempo, que es lo que deja salir N1, y aun trabajando rápido ($\mu_{32} = 3$) solo atiende 0,33. Después de cerrar las llegadas, vaciar la nave principal toma casi 200 unidades de tiempo.

5.4. Escenario B

$\lambda = 3$; N1: $N(3,0; 1,0)$; N2: exponencial con $\mu_2 = 1,5$; N3: $N(4,0; 2,0)$ si $n_3 < 5$ y $N(2,0; 1,5)$ si $n_3 \geq 5$.

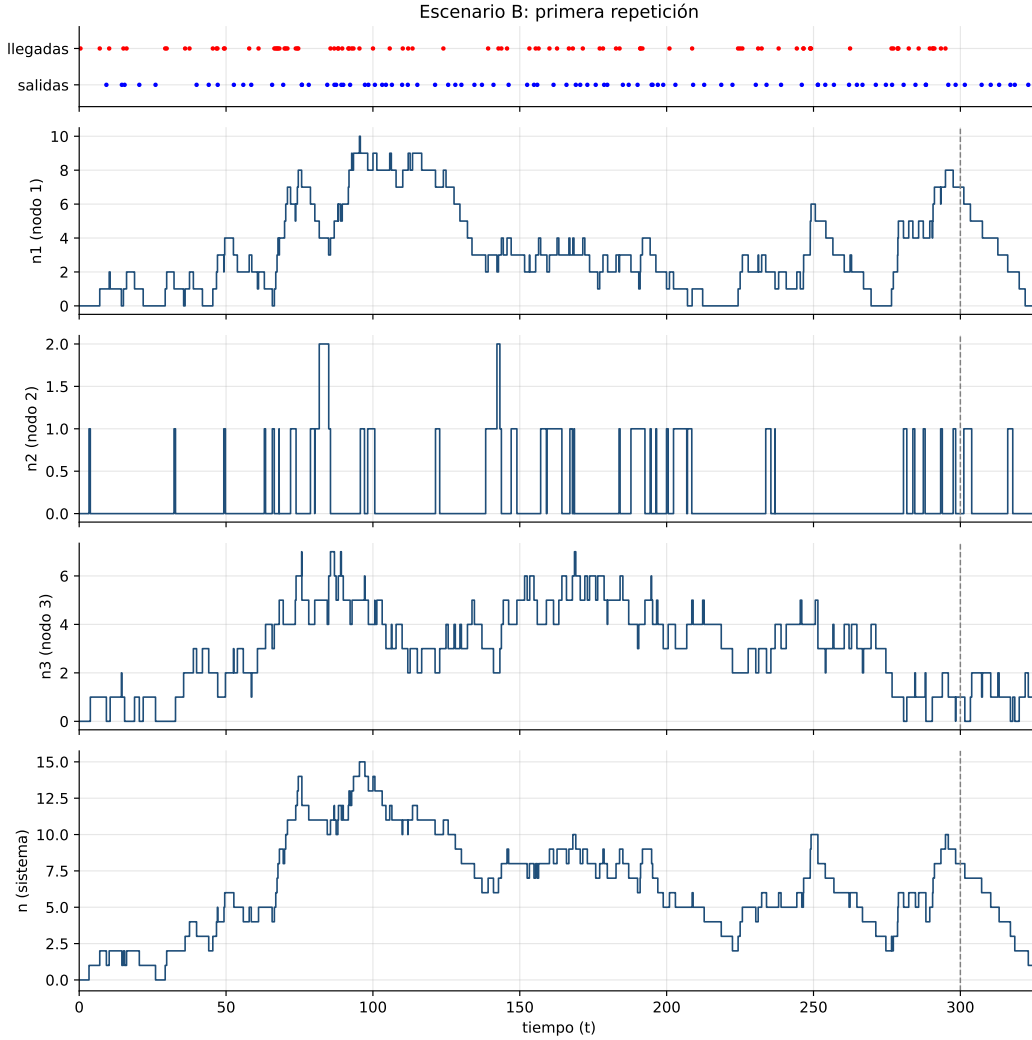


Figura 5: Escenario B, primera repetición.

El comportamiento es muy distinto. N3 ya no se desborda: pasa cerca del 80 % del tiempo con entre 2 y 6 vehículos. El nodo que manda ahora es N1, con $\rho_1 = 3,0/3,0 = 1$, y en la figura 5 tiene las subidas y bajadas largas típicas de una cola en el límite de la estabilidad.

5.5. Veinte repeticiones por escenario

Para cada medida se calcularon la media muestral, la desviación estándar muestral y el intervalo de confianza del 95 % con la distribución t de Student:

$$\bar{x} \pm t_{0,975;19} \frac{s}{\sqrt{20}}, \quad t_{0,975;19} = 2,093.$$

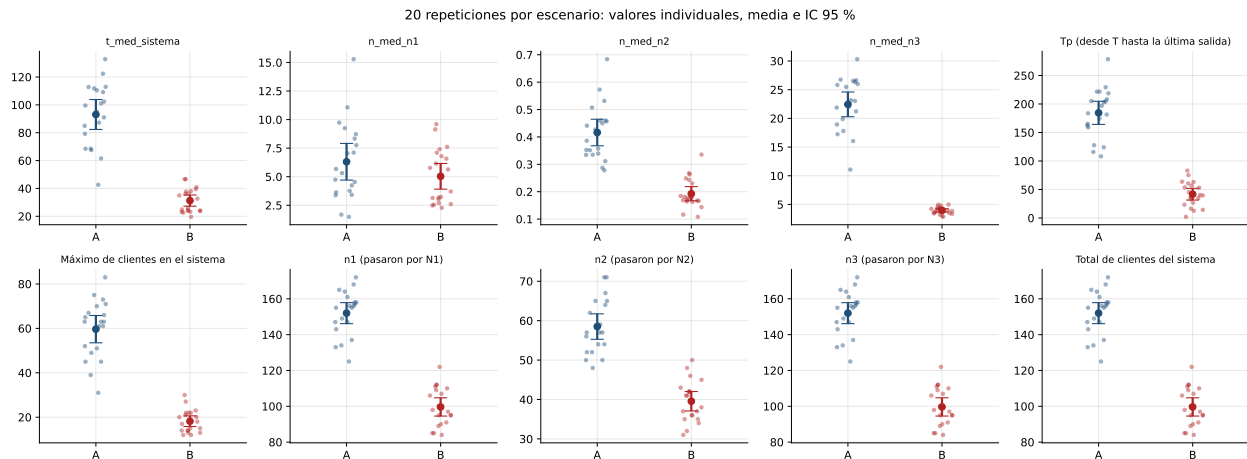
La última columna de las tablas 6 y 7 es el semiancho del intervalo como porcentaje de la media, que sirve para comparar la precisión de medidas con escalas distintas.

Tabla 6: Escenario A: 20 repeticiones.

Medida	Media	Desv. estándar	IC 95 %	Semiancho
t_med_sistema	93,1046	22,9643	[82,357; 103,852]	11,5 %
n_med_n1	6,3089	3,4278	[4,705; 7,913]	25,4 %
n_med_n2	0,4162	0,1035	[0,368; 0,465]	11,6 %
n_med_n3	22,4450	4,6100	[20,287; 24,603]	9,6 %
Tiempo desde T hasta la última salida	184,4745	43,3683	[164,177; 204,771]	11,0 %
Máximo de clientes en el sistema	59,6500	13,1360	[53,502; 65,798]	10,3 %
n_1 (pasaron por N1)	152,0000	12,5153	[146,143; 157,857]	3,9 %
n_2 (pasaron por N2)	58,5000	6,9016	[55,270; 61,730]	5,5 %
n_3 (pasaron por N3)	152,0000	12,5153	[146,143; 157,857]	3,9 %
Total del sistema	152,0000	12,5153	[146,143; 157,857]	3,9 %

Tabla 7: Escenario B: 20 repeticiones.

Medida	Media	Desv. estándar	IC 95 %	Semiancho
t_med_sistema	31,2736	8,5904	[27,253; 35,294]	12,9 %
n_med_n1	5,0342	2,4017	[3,910; 6,158]	22,3 %
n_med_n2	0,1927	0,0555	[0,167; 0,219]	13,5 %
n_med_n3	3,9561	0,6266	[3,663; 4,249]	7,4 %
Tiempo desde T hasta la última salida	41,7405	21,9625	[31,462; 52,019]	24,6 %
Máximo de clientes en el sistema	18,2000	5,1463	[15,791; 20,609]	13,2 %
n_1 (pasaron por N1)	99,6500	10,8883	[94,554; 104,746]	5,1 %
n_2 (pasaron por N2)	39,5500	5,2763	[37,081; 42,019]	6,2 %
n_3 (pasaron por N3)	99,6500	10,8883	[94,554; 104,746]	5,1 %
Total del sistema	99,6500	10,8883	[94,554; 104,746]	5,1 %

**Figura 6:** Valores de las 20 repeticiones (puntos), media e intervalo de confianza del 95 % para cada medida y escenario.

Como apoyo al análisis se calculó la intensidad de tráfico aproximada de cada nodo, $\rho_j = \text{tasa de llegada al nodo} \times \text{tiempo medio de servicio}$ (tabla 8).

Tabla 8: Intensidad de tráfico aproximada de cada nodo.

Nodo	Escenario A	Escenario B
ρ_1	1,05	1,00
ρ_2	0,46	0,20
ρ_3 con $n_3 < 5$	1,95	1,33
ρ_3 con $n_3 \geq 5$	1,43	0,67

5.6. ¿Qué puede decir de la media y la desviación estándar de la muestra para cada medida?

Los conteos son las medidas más estables y sirven de validación. Por N1 pasaron $152 \pm 12,5$ vehículos en A y $99,7 \pm 10,9$ en B. Eso es lo que se espera de un proceso de Poisson con $300/2 = 150$ y $300/3 = 100$ llegadas en promedio, cuya desviación teórica es $\sqrt{150} \approx 12,2$ y $\sqrt{100} = 10$. Por N2 pasó el 38,5 % de los vehículos en A y el 39,7 % en B, muy cerca del 40 % del modelo. En los dos escenarios n_3 y el total del sistema coinciden con n_1 , porque todos los vehículos terminan en N3 y la simulación sigue hasta que sale el último.

Las medidas de congestión varían mucho más. El coeficiente de variación está entre 8 y 11 % para los conteos, pero sube a 25 % para el tiempo medio en el sistema de A y a 54 % para el número medio en N1 de A (3,43/6,31). Estas medidas dependen de cómo se juntan las rachas de llegadas y de servicios largos, no solo de cuántos vehículos llegan.

La mayor variabilidad relativa está siempre en N1, que en los dos escenarios trabaja con $\rho_1 \approx 1$. Una cola en ese punto se comporta casi como una caminata aleatoria: puede pasar largos ratos vacía o acumular muchos vehículos según la suerte de la corrida. En la repetición 5 de A, N1 tuvo en promedio 15,3 vehículos y en la 7 solo 1,5.

En N3 se ve el efecto de la regla de los operarios. En A tiene la media más alta (22,4 vehículos) pero una variación relativa baja, porque está siempre saturado: ni con el ritmo rápido alcanza ($\rho_3 = 1,43$). En B pasa algo interesante. Con menos de 5 vehículos N3 sería inestable ($\rho_3 = 1,33$), así que la cola crece; al llegar a 5 los operarios aceleran ($\rho_3 = 0,67$) y la cola baja. N3 termina rondando el umbral, con media 3,96 y la desviación más pequeña de todas las medidas de congestión (0,63). La regla funciona como un regulador.

5.7. ¿Qué análisis puede realizar de los intervalos de confianza del 95 %?

La lectura correcta de un intervalo del 95 % es que, si se repitiera muchas veces el experimento completo de 20 corridas, cerca del 95 % de los intervalos así contruidos contendrían el valor esperado verdadero de la medida. Como los dos escenarios tienen algún nodo inestable, no existe un régimen estable que estimar; lo que se estima es el comportamiento de un periodo de 300 unidades de tiempo que arranca vacío.

La precisión es muy desigual. El semiancho relativo va de 3,9 % para el número de vehículos en A a 25,4 % para el número medio en N1 en A. Para el tiempo medio en el sistema de A el intervalo es [82,36; 103,85], con un semiancho de 10,7, el 11,5 % de la media. Bajar ese error al 5 % exigiría unas $n \approx (2,093 \cdot 22,96 / (0,05 \cdot 93,10))^2 \approx 107$ corridas, cinco veces más.

Casi todas las diferencias entre A y B son claras. En 9 de las 10 medidas los intervalos no se traslapan. El tiempo medio en el sistema está entre 82,4 y 103,9 en A y entre 27,3 y 35,3 en B: B es unas tres veces mejor para el usuario, aunque su ventanilla sea más lenta (3,0 contra 2,1).

La mejora viene de que llegan menos vehículos y de que N3, el cuello de botella de A, en B atiende más rápido cuando se congestiona ($\mu_{32} = 2,0$ contra 3,0).

La única medida sin diferencia significativa es el número medio en N1: [4,70; 7,91] en A y [3,91; 6,16] en B. En los dos escenarios N1 está en $\rho_1 \approx 1$, y con esa variabilidad 20 corridas no alcanzan para separar los dos casos. Eso no significa que sean iguales, solo que la muestra no permite afirmar que sean distintos.

Sobre el supuesto de normalidad. El intervalo t supone que la media muestral es aproximadamente normal. Para los conteos se cumple bien. Para el tiempo después de T o el número medio en N1, con colas largas hacia arriba (en B el tiempo después de T fue 12,7 en una repetición y 75,2 en otra), la aproximación con 20 corridas es más dudosa, y esos intervalos conviene tomarlos como orientativos.

6. Numeral 5.3: inventario probabilístico

6.1. El modelo y sus parámetros

Se implementó el modelo de la sección 5.6 de [1]. Un almacén vende un producto a r por unidad. Los clientes llegan según un proceso de Poisson de parámetro λ y cada uno pide una cantidad aleatoria con distribución G ; si no hay suficiente se le vende lo que queda y el resto de la venta se pierde. Cuando el nivel del inventario x queda por debajo de p y no hay pedido pendiente, se piden $P - x$ unidades, que llegan L unidades de tiempo después y se pagan al recibirlas. Además se paga un costo de almacenamiento por unidad y unidad de tiempo. Los sucesos son dos, la llegada de un cliente (t_C) y la llegada de un pedido (t_P), y el programa sigue la misma estructura de los numerales anteriores.

Tabla 9: Parámetros de la simulación.

Símbolo	Significado	Valor
P_0	inventario inicial	20
P	nivel máximo de inventario	100
p	nivel por debajo del cual se hace un pedido	20
L	tiempo que tarda en llegar un pedido	5
h	valor a pagar por unidad, $c(y) = h y$	100
r	precio de venta al público	150
λ	tasa de llegada de clientes	1
G	demanda de cada cliente	uniforme en $\{1, \dots, 5\}$
a	costo de almacenamiento por unidad y unidad de tiempo	1
T	horizonte de simulación	100

Los seis primeros valores son los de la guía. Hay que aclarar dos cosas sobre ellos. La guía describe p como la “cantidad de producto a pedir si no hay pedidos pendientes”, pero en el modelo del libro p es el **nivel de reorden** y la cantidad pedida es $P - x$; así aparece también en la Figura 1 de la guía, tomada de la Figura 5.9 del libro, donde los pedidos se marcan como $P - x_1$ y $P - x_2$. Se siguió el libro. La otra es que en el libro h es el costo de almacenamiento y el costo del pedido es una función aparte $c(y)$; como la guía define $h = 100$ como el valor a pagar por unidad, se usó $c(y) = 100 y$ y, para el almacenamiento, que la guía no da, $a = 1$.

La guía deja λ sin valor y tampoco fija G ni T . Para la simulación principal se tomó $\lambda = 1$, demanda uniforme entre 1 y 5 unidades (media 3) y $T = 100$, y al final se repitió el experimento con varios valores de λ .

6.2. Correcciones al pseudocódigo del libro

Al implementar el pseudocódigo aparecieron tres detalles:

1. En **Llegada_cliente** la condición de venta completa está escrita como **demanda <x**, pero el texto dice “mayor o igual”. Con $<$, un cliente que pide exactamente lo que queda se contaría como no satisfecho. Se usó $<=$.
2. En **Llegada_pedido** aparece **Si var_aux >0**, una variable que no está definida; se entiende que es **var_nivel0**.
3. El libro guarda **var_nivel0 = t** cada vez que un cliente encuentra el inventario insuficiente, también cuando ya estaba en cero, con lo que se pierde el tramo anterior. Aquí se guarda solo cuando el inventario pasa a cero, incluso si llega a cero con una venta completa.

También se agregó el cierre del periodo: si el último suceso ocurre antes de T , se acumula el almacenamiento y el tiempo en cero hasta T , y el porcentaje de inventario en cero se calcula sobre $\max(T, t_{final})$.

6.3. Resultados de la simulación principal

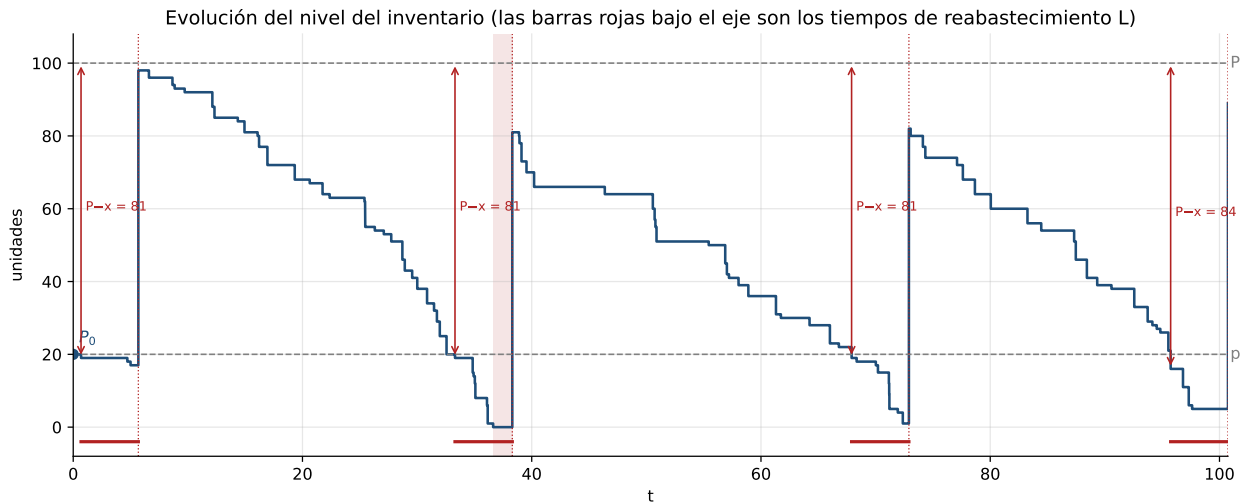


Figura 7: Evolución del nivel del inventario, al estilo de la Figura 1 de la guía. Las flechas son los pedidos de $P - x$ unidades, las barras bajo el eje el tiempo de reabastecimiento L y la franja sombreada el periodo con el inventario en cero.

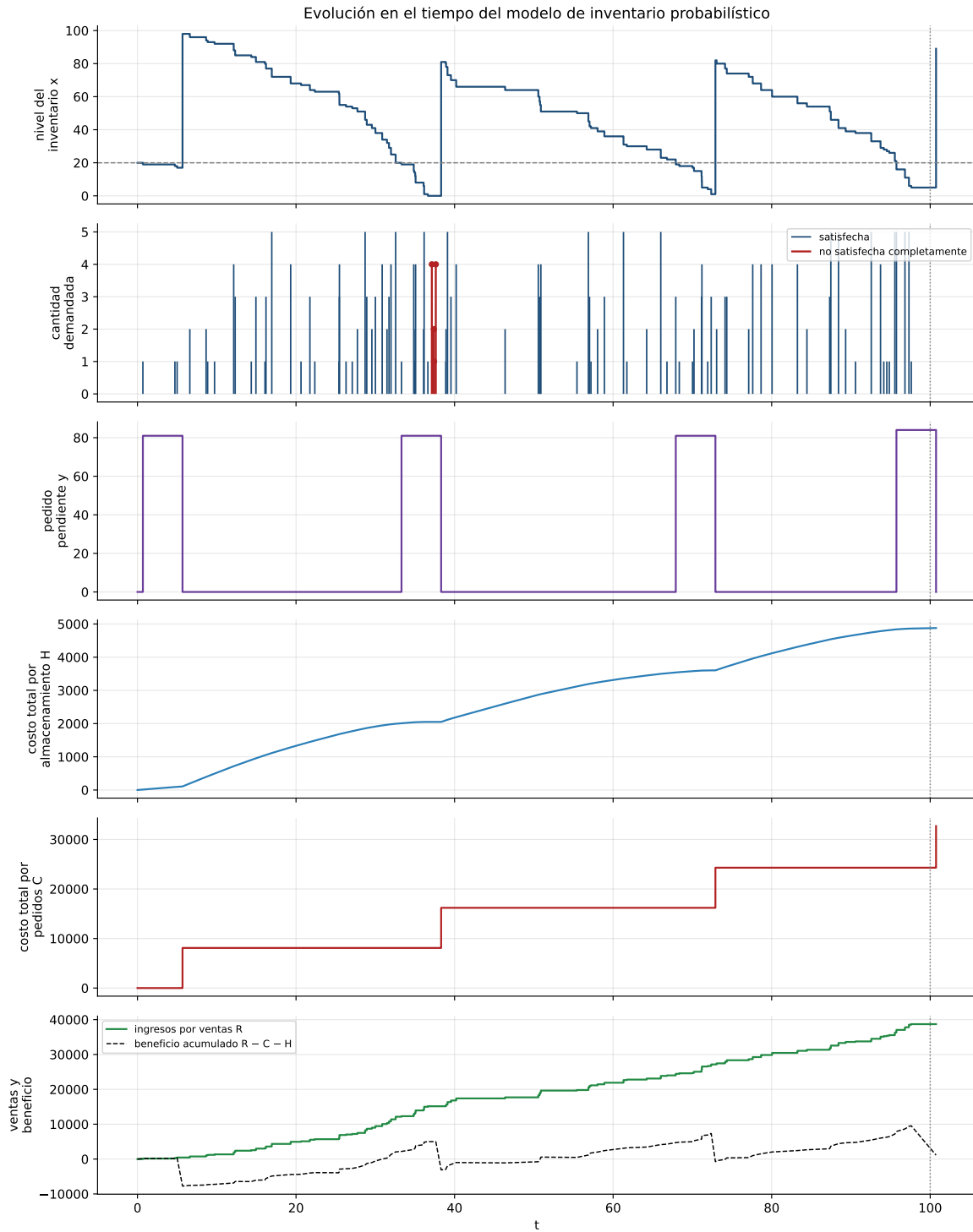


Figura 8: Evolución en el tiempo de las seis variables que pide la guía: nivel del inventario, cantidad demandada por cada cliente (en rojo las que no se satisficieron), pedido pendiente, costo total por almacenamiento, costo total por pedidos, e ingresos por ventas con el beneficio acumulado.

Tabla 10: Medidas de desempeño al final de la simulación principal.

Medida	Valor
Beneficio $R - C - H$	1.121,70
Porcentaje de clientes satisfechos	95,96 % (95 de 99)
Porcentaje de inventario en cero	1,66 %
Ingresos por ventas R (258 unidades)	38.700,00
Costo por pedidos C (4 pedidos: 81, 81, 81 y 84 unidades)	32.700,00
Costo por almacenamiento H	4.878,30
Inventario final	89 unidades
Beneficio ajustado por inventario final	8.021,70

El beneficio de 1.121,70 parece bajo, y la razón se ve en el último panel de la figura 8: el cuarto pedido se hizo en $t = 95,74$ y llegó en $t = 100,74$, cuando el periodo ya había cerrado. Se pagaron 8.400 por unidades que no se alcanzaron a vender, y la simulación terminó con 89 unidades en bodega. Para separar ese efecto de borde se calculó un *beneficio ajustado*, $R - C - H + h(x_{final} - P_0)$, que valora a costo la diferencia entre el inventario final y el inicial; en esta corrida es 8.021,70.

El único desabastecimiento ocurrió entre $t = 37,13$ y $t = 38,31$. El segundo pedido se hizo en $t = 33,31$ con 19 unidades en bodega, y en las cinco unidades de tiempo que tardó en llegar la demanda se las llevó todas. Los cuatro clientes no satisfechos llegaron en ese hueco. Los otros tres pedidos llegaron cuando aún quedaban 17, 1 y 5 unidades.

6.4. Réplicas y efecto de la tasa de llegada

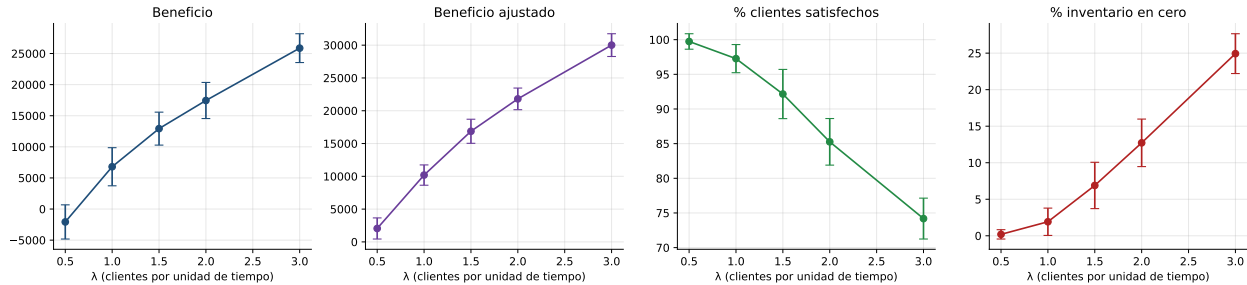
El libro cierra la sección indicando que hay que replicar la simulación para estimar las medidas. Se hicieron 30 réplicas con $\lambda = 1$ (tabla 11) y, como la guía deja λ libre, otras 30 para cada $\lambda \in \{0,5; 1; 1,5; 2; 3\}$ (tabla 12 y figura 9).

Tabla 11: 30 réplicas con $\lambda = 1$ y $T = 100$.

Medida	Media	Desv. estándar	IC 95 %
Beneficio $R - C - H$	6.799,61	3.063,02	[5.655,86; 7.943,36]
Beneficio ajustado	10.189,61	1.549,03	[9.611,20; 10.768,03]
Clientes satisfechos (%)	97,27	2,03	[96,51; 98,02]
Inventario en cero (%)	1,92	1,87	[1,22; 2,62]
Ingresos por ventas R	44.390,00	4.228,34	[42.811,11; 45.968,89]
Costo por pedidos C	32.983,33	2.142,52	[32.183,30; 33.783,36]
Costo por almacenamiento H	4.607,05	249,08	[4.514,04; 4.700,06]
Clientes atendidos	101,40	10,42	[97,51; 105,29]
Pedidos realizados	4,00	0,26	[3,90; 4,10]
Inventario final	53,90	18,36	[47,04; 60,76]

Tabla 12: Promedio de 30 réplicas para distintos valores de λ .

λ	Beneficio	Benef. ajustado	Satisfechos (%)	Inventario en cero (%)	Pedidos
0,5	-2.072	2.048	99,74	0,20	2,33
1,0	6.800	10.190	97,27	1,92	4,00
1,5	12.926	16.869	92,16	6,89	5,57
2,0	17.449	21.809	85,26	12,72	6,73
3,0	25.865	30.008	74,19	24,93	8,57


Figura 9: Medidas de desempeño en función de λ (media $\pm$ una desviación estándar, 30 réplicas).

6.5. Análisis

Una sola corrida no basta para evaluar el beneficio. En las 30 réplicas con $\lambda = 1$ el beneficio fue $6.799,61 \pm 3.063,02$, con un intervalo del 95 % de $[5.655,86; 7.943,36]$, y la corrida principal quedó muy por debajo por el pedido de fin de periodo. Buena parte de la variabilidad viene de ese efecto de borde: el beneficio ajustado tiene la mitad de desviación (1.549,03). Esto sugiere que la política debería considerar el horizonte, por ejemplo no hacer pedidos si faltan menos de L unidades de tiempo para cerrar.

Con $\lambda = 1$ la política ($p = 20, P = 100$) da un buen servicio: 97,27 % de clientes satisfechos (IC: 96,51 a 98,02) y el inventario en cero solo el 1,92 % del tiempo. Durante las $L = 5$ unidades de tiempo de espera de un pedido se demandan en promedio $\lambda L E[D] = 1 \cdot 5 \cdot 3 = 15$ unidades, menos que las 20 con que se dispara el pedido. Solo cuando la demanda de ese plazo sale alta hay faltantes, como pasó en la corrida principal.

El efecto de λ confirma ese razonamiento. La demanda esperada durante el plazo de entrega es 15λ , que supera el punto de reorden cuando $\lambda > 4/3$. La tabla 12 muestra el quiebre justo ahí: el tiempo con inventario en cero pasa de 1,9 % con $\lambda = 1$ a 6,9 % con $\lambda = 1,5$, 12,7 % con $\lambda = 2$ y 24,9 % con $\lambda = 3$, y los clientes satisfechos bajan de 97,3 % a 74,2 %. El beneficio sí crece con λ porque se venden más unidades con un margen de 50 cada una, pero con $\lambda = 3$ uno de cada cuatro clientes se va sin su pedido completo.

Con poca demanda el negocio pierde. Con $\lambda = 0,5$ el beneficio medio fue negativo (-2.072). Se vendieron en promedio 152 unidades por 22.780, pero los pedidos costaron 19.307 y el almacenamiento 5.546, porque cada pedido de unas 80 unidades tarda unas 53 unidades de tiempo en venderse. Además se terminó con 61 unidades sin vender en promedio. Para esa demanda convendría un P más bajo.

En resumen, con los valores de la guía $p = 20$ solo es adecuado si llegan a lo sumo 1,3 clientes por unidad de tiempo. Para demandas mayores habría que subir el punto de reorden al menos a 15λ

más un margen de seguridad, o conseguir un proveedor con menor L . El modelo permite probar esas políticas cambiando dos parámetros, que es justamente el uso que le da el libro: ayudar a redefinir la política de gestión del inventario.

7. Numeral 5.4: simulación de eventos discretos en SimPy

Todo este numeral está en el cuaderno `Lab4_5.4_SimPy_160005002.ipynb`. Aquí se resume cada parte con los fragmentos de código y las salidas más importantes.

7.1. Instalación

SimPy se instala con `pip install simpy` tanto en Colab como en un equipo local. Se trabajó con SimPy 4.1.2 sobre Python 3.13.

7.2. Tutorial de Stefan Scherfke

Se siguió la presentación *Discrete-event simulation with SimPy* (EuroPython 2014) paso a paso, junto con el cuaderno *Discrete-event simulation with SimPy.ipynb* del curso, que traduce los mismos ejemplos:

1. **Funciones generadoras.** `yield` entrega un valor y congela la función; `send()` le devuelve un valor. Con `generator(1)` se obtuvo 2 con `next` y `StopIteration` con valor 4 al enviar 3. Es el mecanismo con que SimPy suspende y reanuda procesos.
2. **Primer modelo: dos relojes.** Dos procesos con `env.timeout(0.5)` y `env.timeout(1)` se intercalan según el tiempo simulado: rápido 0, lento 0, rápido 0.5, lento 1, rápido 1.0, rápido 1.5.
3. **Environment y RealtimeEnvironment.** Con el mismo modelo, 10 unidades simuladas tomaron 0,000s en el primero y 1,000s en el segundo con `factor=0.1`.
4. **Los timeouts dejan pasar el tiempo; los procesos también son eventos.** Un orador espera su turno y habla 30 minutos; un moderador espera a cada orador con `yield env.process(...)` y recibe su valor de retorno.
5. **Interrupciones.** El moderador corta a cada orador a los 30 minutos con `interrupt()`.
6. **Eventos de condición.** `speaker_proc | env.timeout(30)` espera lo que ocurra primero.
7. **Recursos compartidos:** ejemplos cortos con `Resource`, `Container` y `Store`.
8. **Asistente a la conferencia,** con y sin *buffet* limitado.

En los pasos 5 y 6 aparecieron dos detalles que valen la pena. Con la versión de las diapositivas que interrumpe siempre a los 30 minutos, el segundo orador terminó solo en $t = 57$ y el moderador intentó interrumpirlo en $t = 60$; SimPy lanza un `RuntimeError`, porque no se puede interrumpir un proceso terminado. El evento de condición lo resuelve casi siempre, pero no cuando hay **empate**: con la semilla usada, el quinto orador habló exactamente 30 minutos. En $t = 145$ ocurren dos eventos a la vez; la condición se dispara con el *timeout*, que SimPy procesa primero, así que **results** no incluye al orador, pero antes de que el moderador reanude la charla ya terminó, y el error vuelve a aparecer. La versión robusta pregunta además si el proceso sigue vivo:

```
results = yield speaker_proc | env.timeout(30)
if speaker_proc not in results and speaker_proc.is_alive:
    speaker_proc.interrupt('No queda tiempo')
```

En el ejemplo del *buffet* con un solo puesto y 3 minutos por persona, la ventana de $15 - 3 = 12$ minutos solo alcanza para 4 de los 5 asistentes. Los cuatro primeros comen en $t = 93, 96, 99$ y 102 , y el quinto se queda sin comer en las dos sesiones, porque todos terminan las charlas a la vez y SimPy atiende las peticiones en el orden en que se hicieron. En las diapositivas, además, la línea `time_left = LEN_BREAK - (env.now - start)` usa una constante que no existe; debe ser `BREAK_LENGTH`, como ya está en el cuaderno del curso.

7.3. Tutorial *SimPy* in 10 minutes

1. **Conceptos básicos.** Un proceso `car` que alterna 5 unidades estacionado y 2 conduciendo: empieza a conducir en 5, estaciona en 7, conduce en 12 y estaciona en 14.
2. **Interacción entre procesos.** El carro eléctrico espera al proceso `charge`, y un conductor lo interrumpe en $t = 3$; el carro atrapa la excepción `simpy.Interrupt` y sale a conducir en $t = 3$ en vez de en 5.
3. **Recursos compartidos.** Una estación de carga con dos puestos (`simpy.Resource(env, capacity=2)`) y cuatro carros que llegan en $t = 0, 2, 4, 6$ con cargas de 5.

```
def car(env, name, bcs, driving_time, charge_duration):
    yield env.timeout(driving_time)
    with bcs.request() as req:          # pedir un puesto de carga
        yield req                      # esperar hasta obtenerlo
    yield env.timeout(charge_duration)
```

El carro 2 llega en $t = 4$ con los dos puestos ocupados y empieza en $t = 5$, cuando sale el carro 0; el carro 3 llega en 6 y empieza en 7. Cada uno espera exactamente una unidad de tiempo, lo mismo que da la cuenta a mano.

7.4. Ejemplos de SimPy

Los siete ejemplos se implementaron con el código de la documentación oficial y se ejecutaron sin cambios, para poder comparar la salida con la publicada (coincide, porque todos fijan `RANDOM_SEED = 42`). La tabla 13 resume qué modela cada uno y qué se obtuvo.

Tabla 13: Ejemplos de SimPy implementados.

Ejemplo	Qué modela y qué usa	Resultado
Bank Renege	Mostrador único; clientes con paciencia entre 1 y 3 que abandonan. <code>Resource</code> y evento de condición <code>req timeout(patience)</code> .	De 5 clientes, 4 fueron atendidos sin esperar y el cliente 2 abandonó en $t = 13,90$ tras esperar 1,17 mientras el cliente 1 estaba en el mostrador.

Ejemplo	Qué modela y qué usa	Resultado
Carwash	Lavadero con 2 máquinas, lavado de 5 minutos, 4 carros iniciales y uno cada 5 a 9 minutos. Resource y espera de otro proceso.	Los carros 2 y 3 esperan hasta $t = 5$ y el 4 hasta $t = 10$; después la cola desaparece, porque la capacidad (0,4 carros/min) supera las llegadas (0,14 carros/min).
Machine Shop	10 máquinas que se dañan cada 300 min en promedio y un técnico que las repara en 30 min. interrupt() y PreemptiveResource con prioridades.	En 4 semanas cada máquina hizo en promedio 3277,5 piezas, el 81 % de las 4032 que haría sin daños. El técnico recibe $10 \cdot 30/300 = 1$ hora de reparación por hora: está saturado.
Movie Renege	Taquilla con 3 películas de 50 boletas; al agotarse una, todos los que esperaban por ella se van. Resource , evento compartido y condición.	Se agotaron a los 28, 38 y 43 minutos; se fueron de la fila 5, 16 y 5 personas cuando se agotaron.
Gas Station Refueling	2 surtidores con tanque común de 200 L y carrotanque que tarda 300 s. Resource , Container y proceso de control.	El tanque bajó a 12 L en $t = 459$ y se llamó al carrotanque en $t = 460$; llegó en $t = 760$ con 188 L. Los carros 5 y 6 quedaron esperando combustible; el 5 esperó 55 s.
Process Communication	Mensajes uno a uno y uno a muchos. Store como tubería y búfer.	Los mensajes que el consumidor no alcanza a leer esperan en el Store y se marcan como <i>LATE</i> (por ejemplo, el de $t = 65$ leído en 66). En el caso uno a muchos cada consumidor tiene su propio Store .
Event Latency	Cable con retardo de 10 unidades entre emisor y receptor. Store y un proceso por mensaje.	Cada mensaje enviado cada 5 unidades llega exactamente 10 después; el retardo vive en la clase Cable , no en los procesos.

La diferencia con los numerales 5.1 a 5.3 está en el enfoque. Allá se programaba a mano la lista de sucesos, el reloj y la elección del suceso más próximo, lo que se conoce como **programación de eventos**. En SimPy se describe lo que hace cada entidad (llegar, pedir un recurso, esperar, salir) y la librería se encarga del reloj y de la cola de eventos, lo que se conoce como **interacción de procesos**. Por eso un modelo con interrupciones y prioridades como el taller de máquinas cabe en unas 100 líneas.

8. Numeral 5.5: simulación de eventos discretos en AnyLogic

Se instaló AnyLogic 8.9 PLE y se desarrollaron paso a paso los cinco tutoriales de la guía, que vienen en la ayuda del programa (*Help* → *Tutorials*). Los modelos terminados se entregan en la

carpeta Modelos_AnyLogic.

8.1. Bank office model (sistema de colas)

Modela una oficina bancaria con un cajero automático y cuatro cajeros humanos. Se construyó en cuatro fases:

1. **Modelo simple.** Diagrama `Source` → `Queue` → `Delay` (ATM) → `Sink`, con unidades de tiempo en minutos. Llegan 0,3 clientes por minuto, la cola admite 15 y el cajero automático tarda `triangular(0.8, 1.5, 3.5)` minutos.
2. **Animación.** El cajero automático se dibujó como un *Point Node* que cambia de color con `ATM.size() > 0 ? red : green` y la cola como un *Path*. Se añadió una ventana 3D, un tipo de agente *Customer* con figura de persona y el objeto 3D del cajero automático.
3. **Cajeros.** Un `SelectOutput` envía la mitad de los clientes al cajero automático y la otra mitad a un bloque `Service` (cola de 20, servicio `triangular(2.5, 6, 11)` minutos) que toma una unidad de un `ResourcePool` de 4 cajeros. La animación usa nodos rectangulares con atractores.
4. **Estadísticas.** Dos gráficas de barras con `ATM.statsUtilization.mean()` y `queue.statsSize.mean()`, y un histograma del tiempo en el sistema medido entre un `TimeMeasureStart` y un `TimeMeasureEnd`.

Hay un detalle de los parámetros. El tutorial dice que el tiempo del cajero automático tiene “media” 1,5, pero según la ayuda de AnyLogic `triangular(0.8, 1.5, 3.5)` usa 1,5 como moda, así que la media real es $(0,8 + 1,5 + 3,5)/3 = 1,93$ minutos. Con eso se pueden anticipar las utilizaciones: en la fase 1 el cajero automático recibe 0,3 clientes por minuto y su utilización debería rondar $0,3 \cdot 1,93 = 0,58$. En la fase 3 recibe la mitad, así que debería bajar a cerca de 0,29, y cada cajero humano tendría $0,15 \cdot 6,5/4 \approx 0,24$.

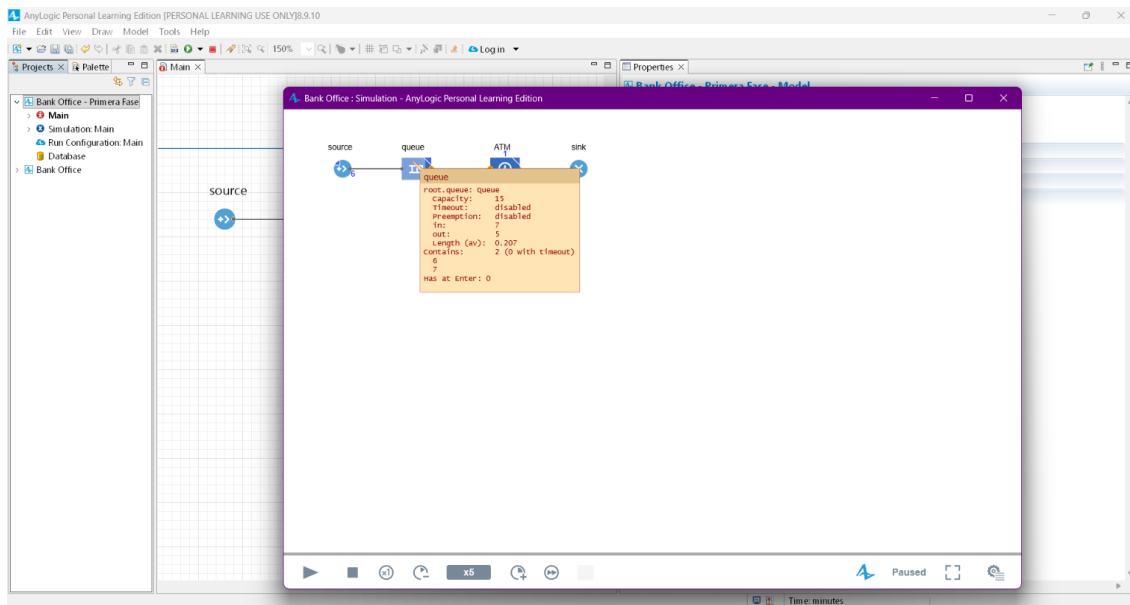


Figura 10: Bank office, fase 1: diagrama de flujo en ejecución.

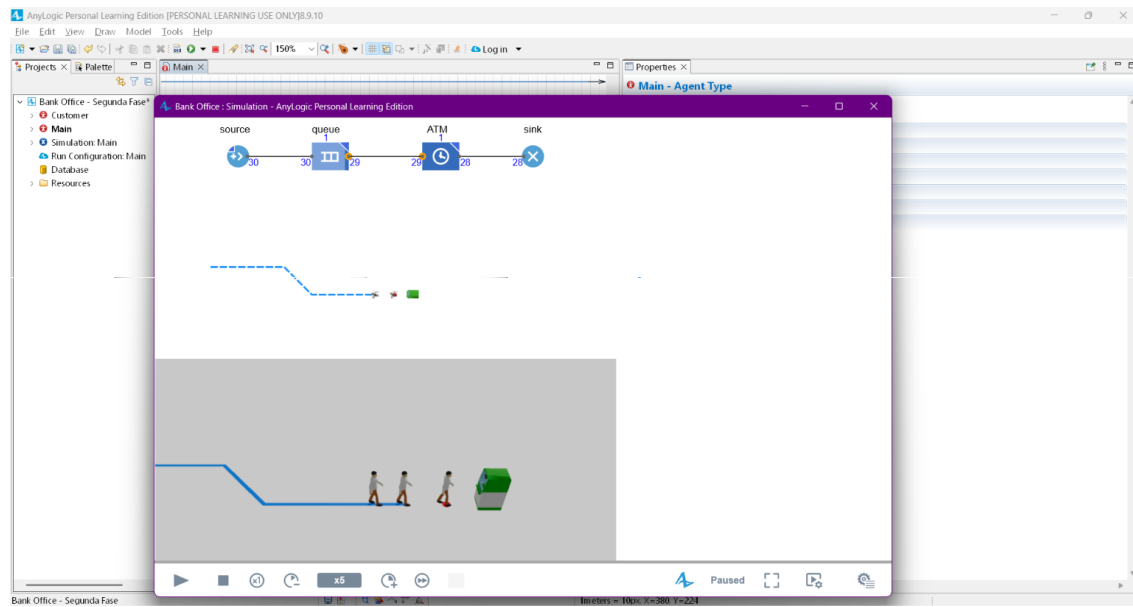


Figura 11: Bank office, fase 2: animación 3D del cajero automático y la cola.

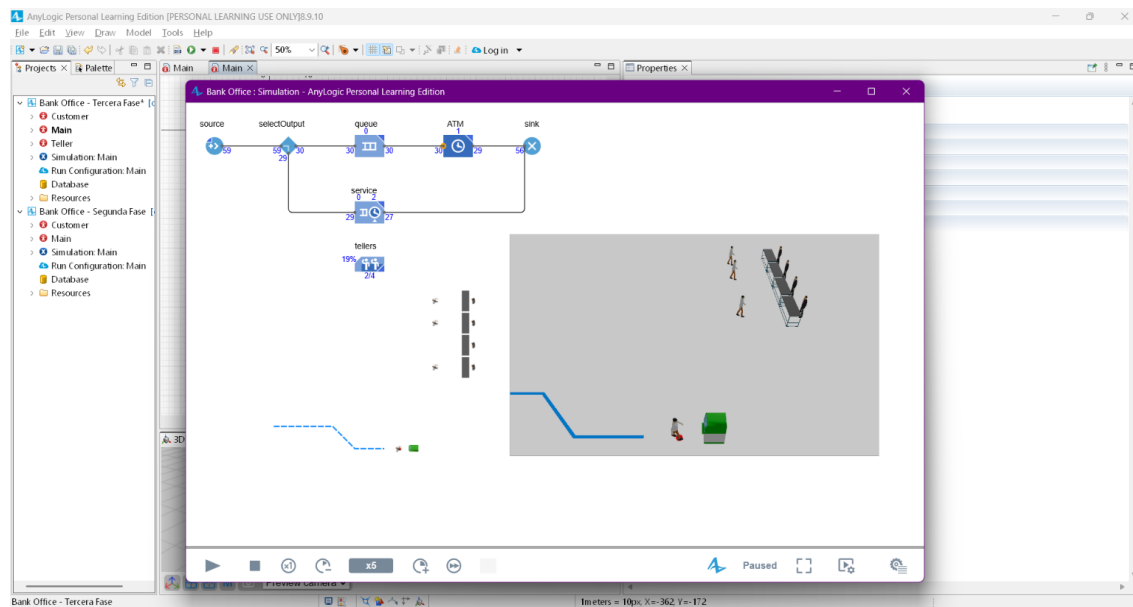


Figura 12: Bank office, fase 3: modelo con cajeros automáticos y humanos.

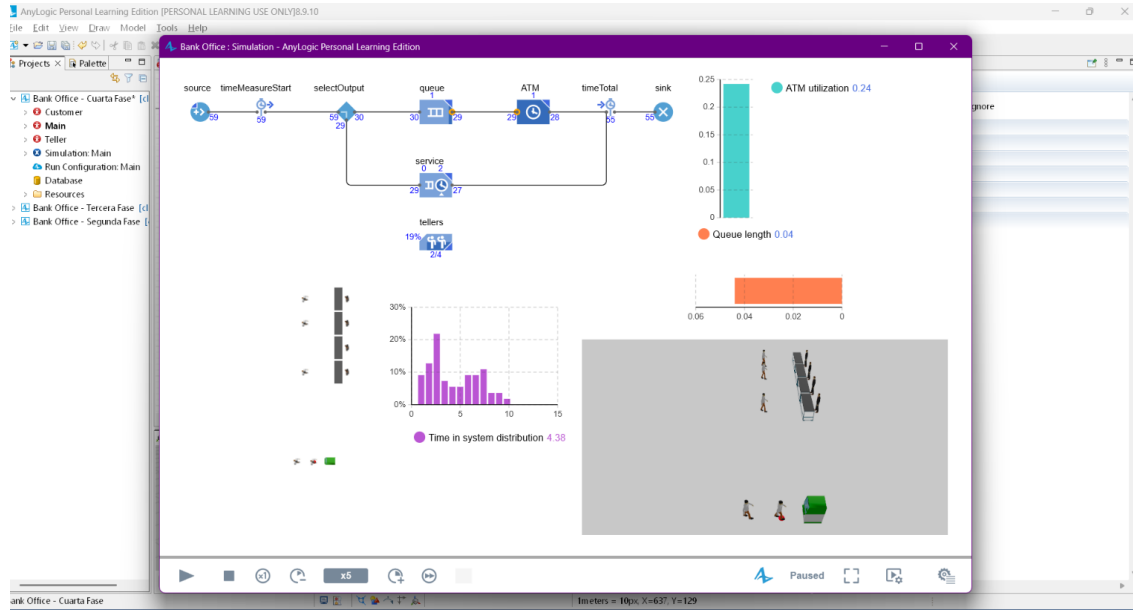


Figura 13: Bank office, fase 4: utilización del cajero automático, longitud media de la cola e histograma del tiempo en el sistema.

En las capturas se ve que cada fase hace lo que debe. En la fase 3 el `selectOutput` repartió 59 clientes en 30 hacia el cajero automático y 29 hacia los cajeros, que es la mitad esperada con probabilidad 0,5. En la fase 4, con esos mismos 59 clientes (unas tres horas simuladas), las gráficas dieron una utilización del cajero automático de 0,24, una longitud media de la cola de 0,04 y un tiempo medio en el sistema de 4,38 minutos.

Esas cifras se pueden contrastar con la teoría. Para la cola del cajero automático, que recibe $\lambda = 0,15$ clientes por minuto, la fórmula de Pollaczek–Khinchine de la cola M/G/1 da

$$L_q = \frac{\lambda^2 E[S^2]}{2(1 - \rho)} = \frac{0,15^2 \cdot 4,064}{2(1 - 0,29)} \approx 0,064,$$

donde $E[S^2] = \text{Var}(S) + E[S]^2 = 0,327 + 1,933^2$ para la triangular (0,8; 1,5; 3,5). El tiempo en el sistema esperado es el promedio de las dos ramas: $1,93 + L_q/\lambda \approx 2,36$ minutos por el cajero automático y unos 6,5 por los cajeros, donde casi no se espera porque cada uno trabaja al 24 %. Eso da $0,5 \cdot 2,36 + 0,5 \cdot 6,5 \approx 4,43$ minutos, muy cerca de los 4,38 simulados. La utilización (0,24 contra 0,29) y la cola (0,04 contra 0,064) quedaron algo por debajo, lo que es normal con una corrida de solo 59 clientes. El histograma muestra las dos ramas: un pico alrededor de 2 a 3 minutos, de los clientes del cajero automático, y una masa más ancha entre 4 y 10 minutos, de los que pasan por los cajeros.

8.2. Subway entrance (peatones)

Modela el vestíbulo de entrada de una estación de metro con la *Pedestrian Library*, en cuatro fases:

1. **Flujo simple.** Sobre la imagen del plano se dibujaron las paredes, una línea de entrada y una de destino. El diagrama `PedSource` → `PedGoTo` → `PedSink` genera 4000 pasajeros por hora que cruzan el vestíbulo.

2. **Torniquetes.** Se dibujó un *Service With Lines* lineal con los torniquetes y se agregó un bloque *PedService* (atFareGates) con tiempo uniforme entre 2 y 3 segundos.
3. **Mapa de densidad.** Un *Density map* muestra con colores dónde se aglomeran los pasajeros.
4. **Máquinas expendedoras.** Un *PedSelectOutput* manda el 10 % de los pasajeros, los que no tienen tiquete, a un *PedService* con máquinas expendedoras de tiempo triangular(7, 12, 40) segundos antes de los torniquetes.

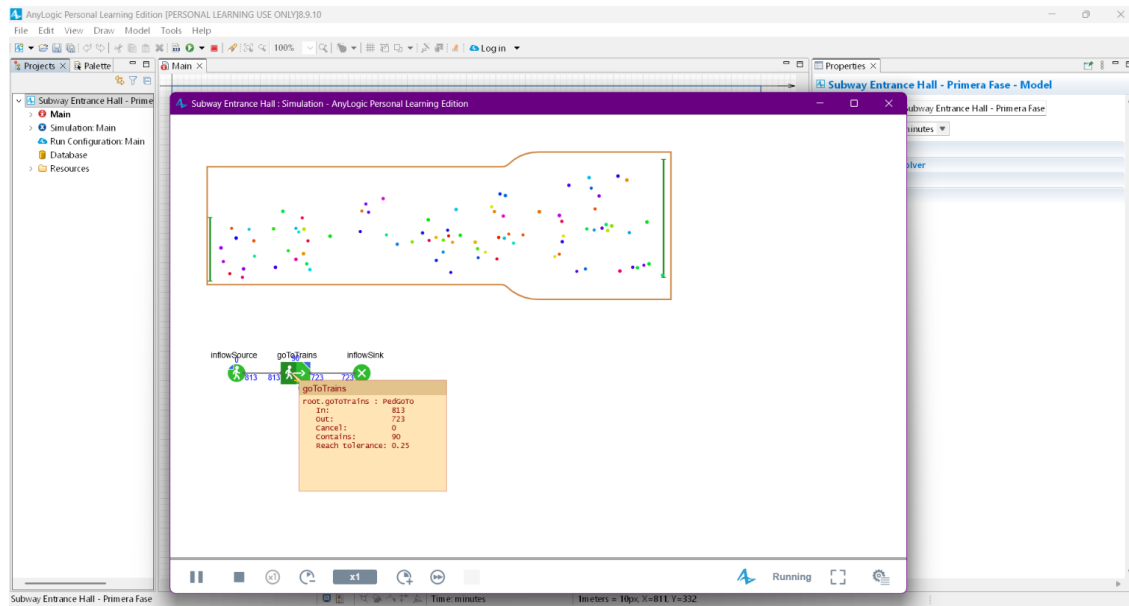


Figura 14: Subway entrance, fase 1: flujo simple de peatones.

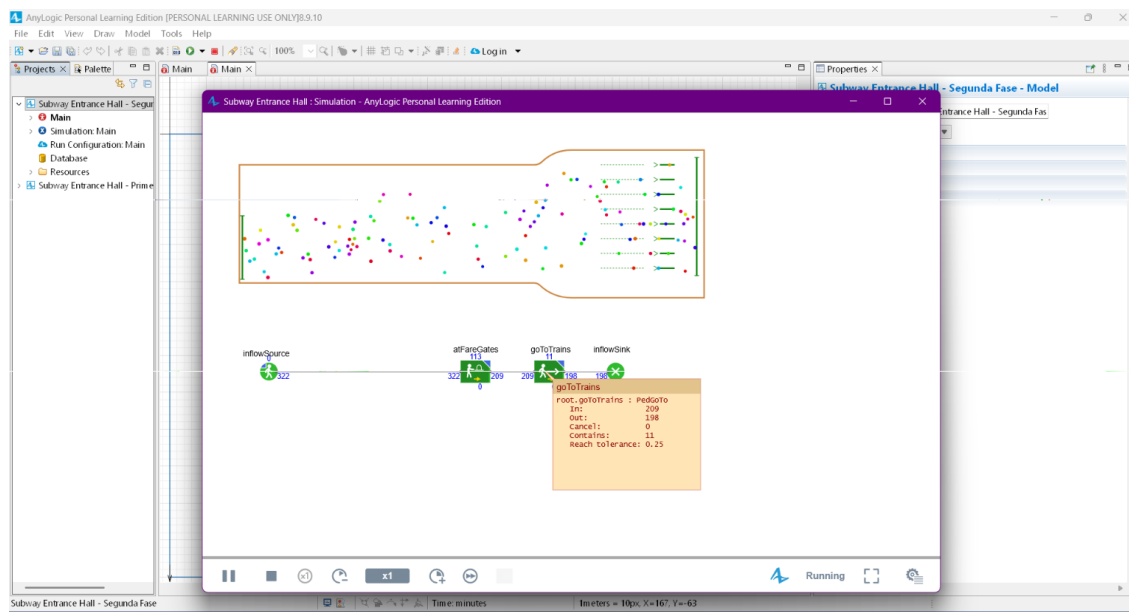


Figura 15: Subway entrance, fase 2: filas en los torniquetes.

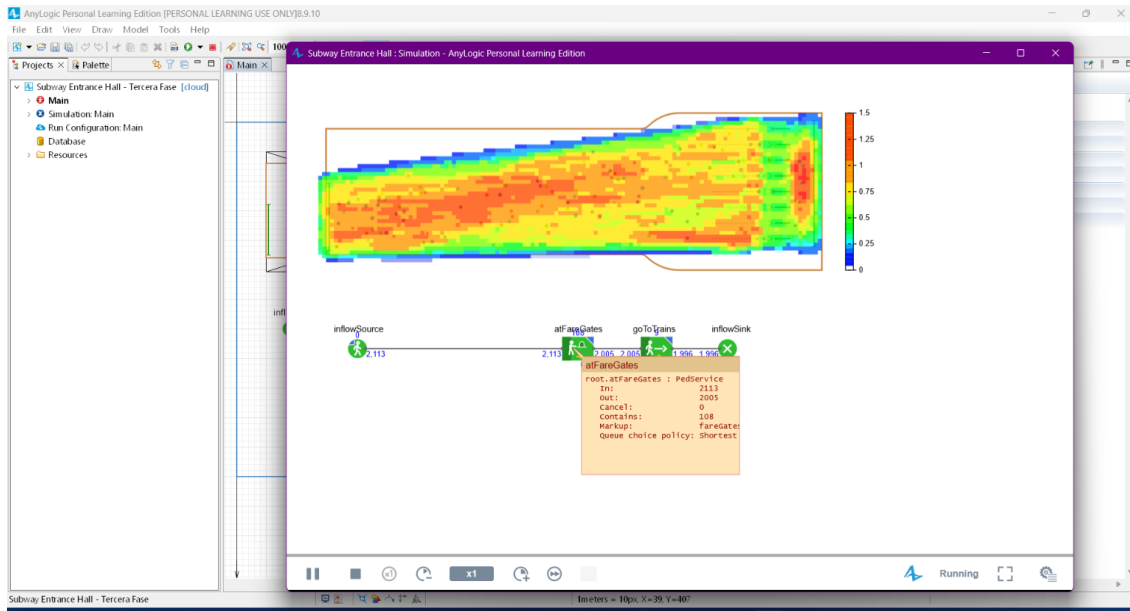


Figura 16: Subway entrance, fase 3: mapa de densidad de peatones.

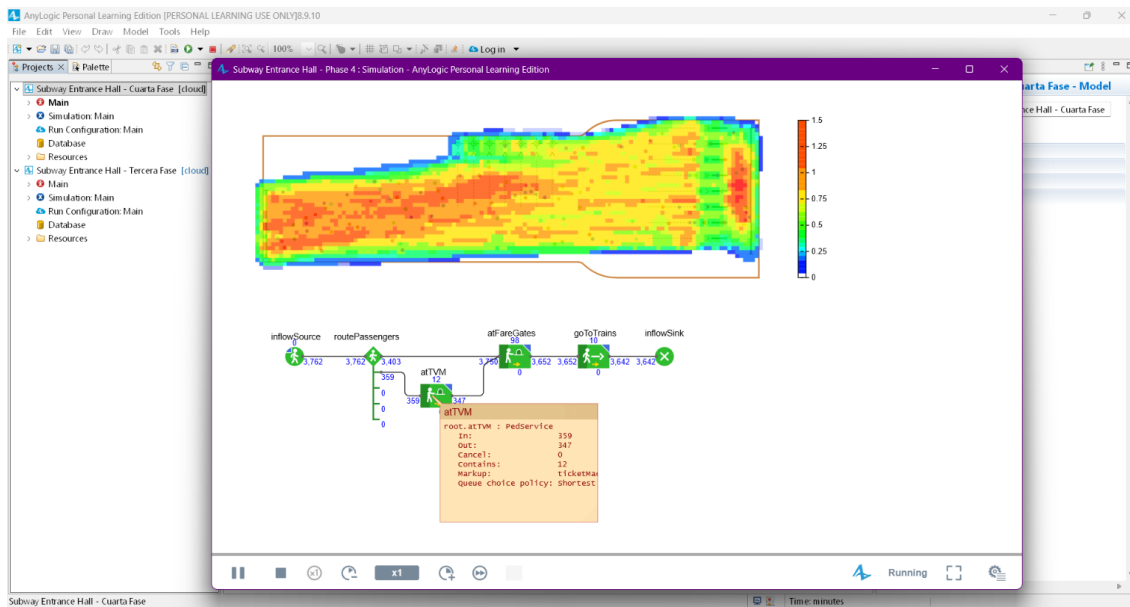


Figura 17: Subway entrance, fase 4: modelo completo con máquinas expendedoras.

En la fase 1, sin ningún obstáculo, los pasajeros cruzan el vestíbulo de la línea de entrada a la de destino sin detenerse: en la captura se habían generado 813 y en ese instante había 90 caminando. Al poner los torniquetes en la fase 2 aparecen las filas. Dentro del bloque `atFareGates` llegó a haber 113 personas entre las que hacían fila y las que pasaban, y en la fase 3 se mantuvo alrededor de 108 después de más de 2000 pasajeros, así que la fila no se disolvía. El mapa de densidad muestra dónde está el problema: la franja roja, con más de un pasajero por metro cuadrado, queda justo antes de los torniquetes, y la parte media del vestíbulo también se carga porque la gente se va represando desde allí. Es la misma conclusión del tutorial: con 4000 pasajeros por hora, el número de torniquetes del modelo apenas alcanza.

En la fase 4 el `PedSelectOutput` desvió a las máquinas 359 de 3762 pasajeros, el 9,5 %, muy cerca del 10 % configurado. Las máquinas tenían 12 personas en ese momento. Como su tiempo de servicio, triangular de 7 a 40 segundos con moda 12, tiene una media de 19,7 segundos, mucho mayor que los 2 a 3 segundos de un torniquete, una fracción pequeña de pasajeros basta para formar una fila propia. En el mapa de esta fase aparece una zona ocupada nueva, en la parte superior del vestíbulo donde están las máquinas, y el total de pasajeros que llega a los torniquetes no cambia, porque los que compran tiquete igual terminan pasando por ellos.

8.3. Intersection (tráfico vehicular)

Modela una intersección real (Tapiolavagen y Menninkaisentie) con la *Road Traffic Library*, con unidades de tiempo en segundos:

1. **Carretera.** Sobre la imagen del plano se dibujó la vía y un diagrama `CarSource` → `CarMoveTo` → `CarDispose`.
2. **Animación 3D** con un tipo de agente `Car` con figura 3D.
3. **Intersección.** Al conectar la segunda vía se crea la intersección automáticamente. Los bloques `SelectOutput` dejan seguir al 70 % de los carros por la vía principal y desvían al 30 % hacia la vía secundaria.
4. **Parqueadero.** Un *Parking lot* al lado de la vía, al que va el 10 % de los carros, con un `Delay` de capacidad 5 que representa el tiempo estacionado.
5. **Buses.** Un segundo `CarSource` genera 20 buses por hora de 10 metros que paran en un *Bus stop* un tiempo `triangular(15, 30, 90)` segundos.
6. **Semáforos.** Un bloque `TrafficLight` con fases de 30 segundos para la vía principal y 20 para la secundaria, y un `RoadNetworkDescriptor` con mapa de densidad para ver los trancones.

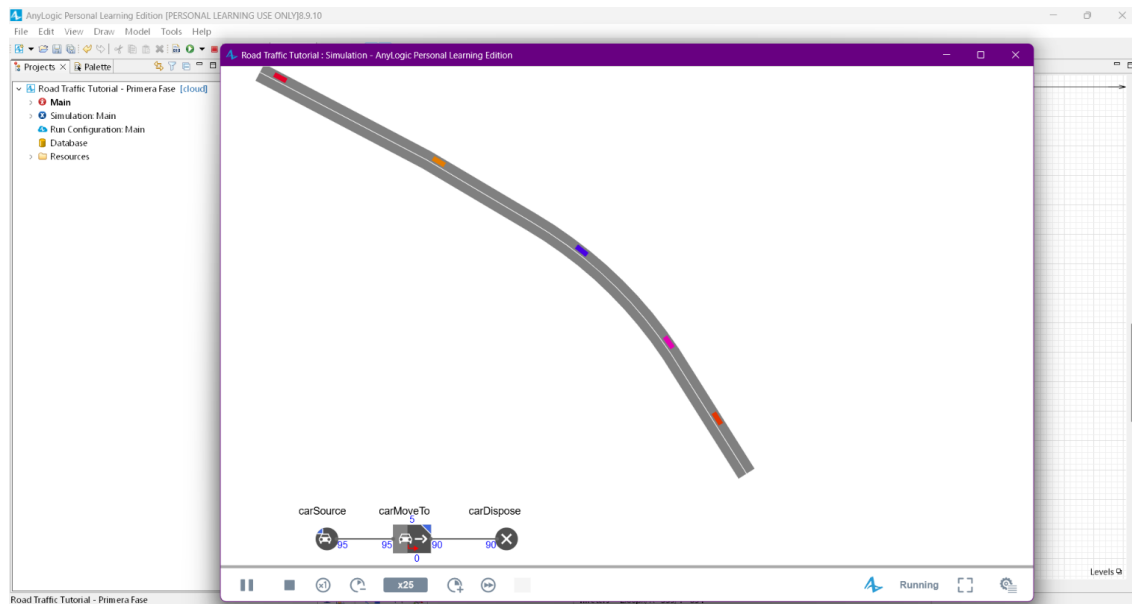


Figura 18: Intersection, fase 1: carros recorriendo la vía en 2D.

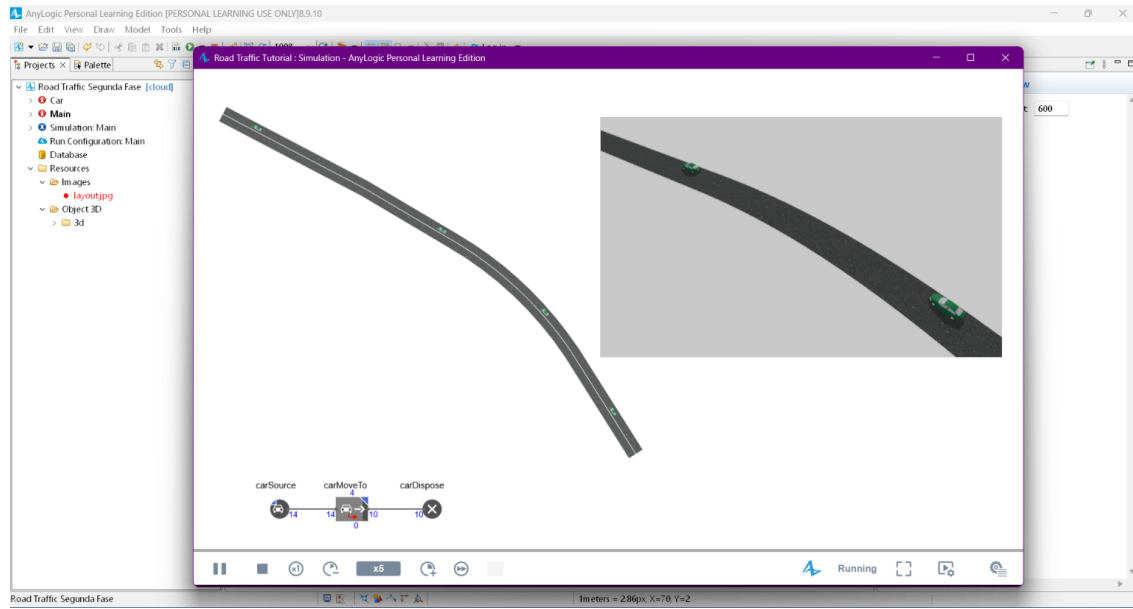


Figura 19: Intersection, fase 2: la misma vía con animación 3D.

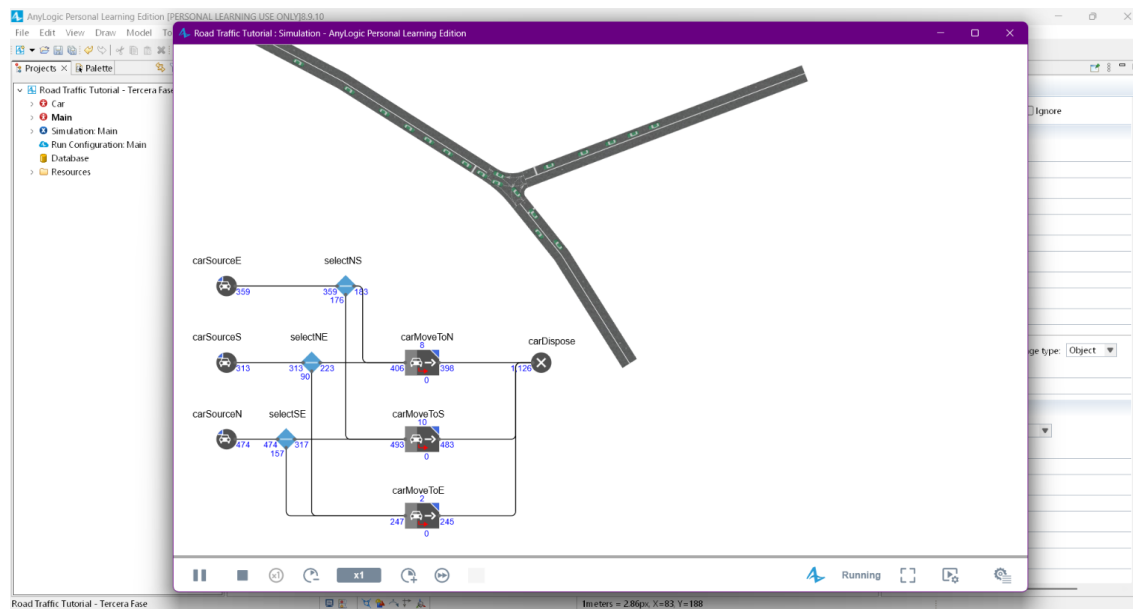


Figura 20: Intersection, fase 3: intersección y diagrama de flujo con los desvíos.

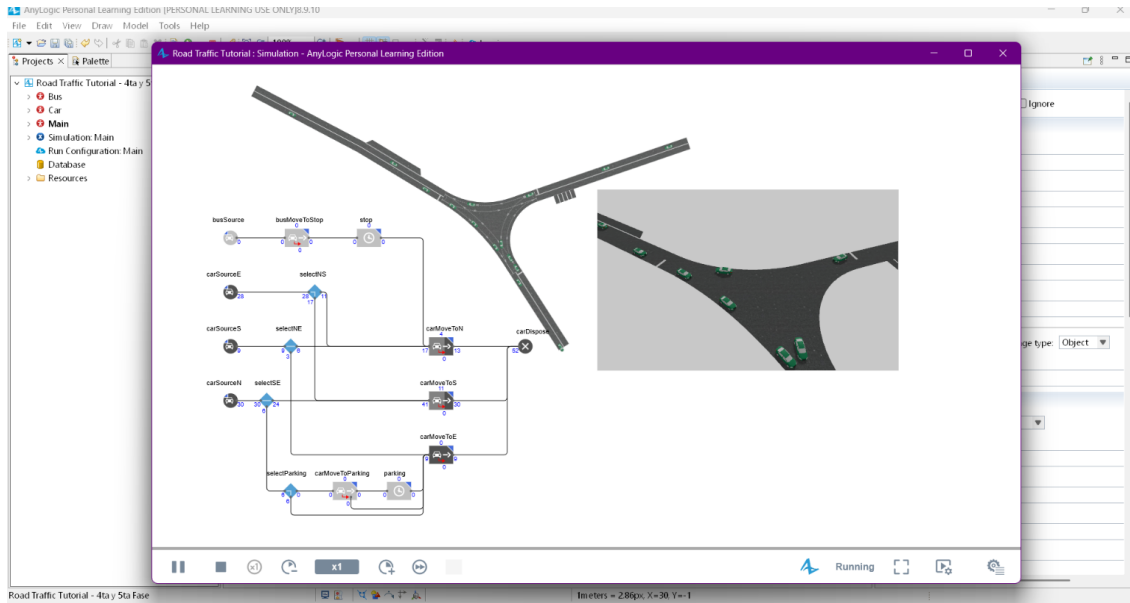


Figura 21: Intersection, fases 4 y 5: parqueadero y buses.

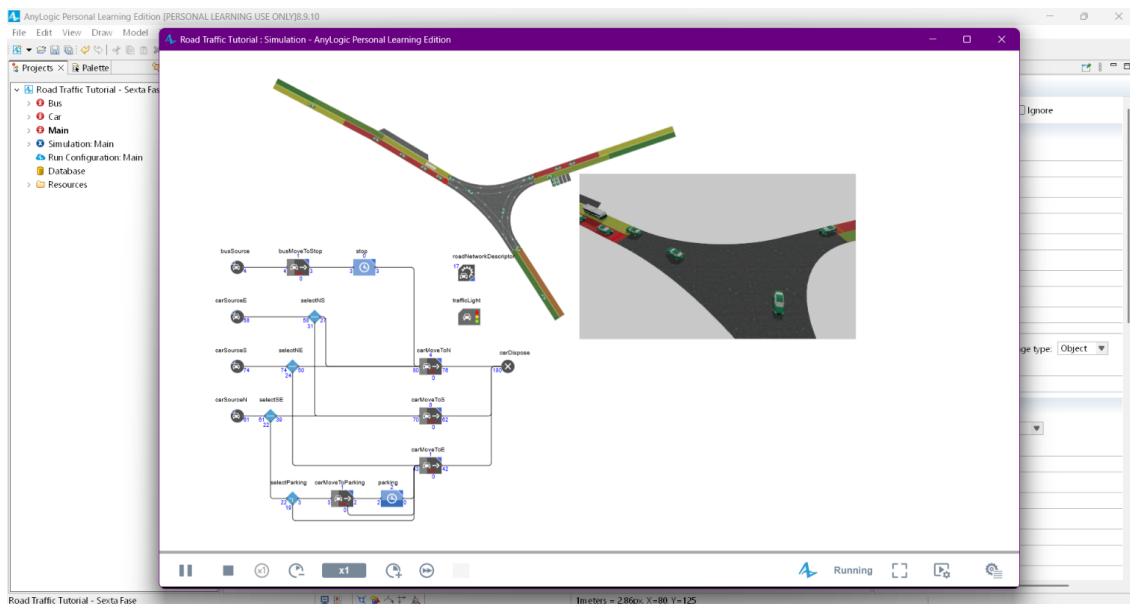


Figura 22: Intersection, fase 6: semáforos y mapa de densidad del tráfico.

En la fase 1 la vía funciona como una cinta: en la captura se habían generado 95 carros, 90 ya habían llegado al final y 5 iban en camino, sin ninguna interacción entre ellos. La fase 3 es la que le da sentido al modelo. Los dos **SelectOutput** de la vía principal repartieron el tráfico casi en la proporción configurada: de los 313 carros que venían del sur, 223 siguieron al norte (71 %) y 90 giraron al este; de los 474 que venían del norte, 317 siguieron al sur (67 %) y 157 giraron. Los carros de la vía secundaria se dividieron por mitades (183 y 176). Con 1126 carros ya despachados, en la animación se ve cómo los que giran tienen que esperar un hueco en el tráfico contrario dentro de la intersección.

Con el parqueadero y los buses (fases 4 y 5) aparecen otros dos tipos de retraso: los carros que

entran a estacionar ocupan el carril mientras maniobran, y el bus se detiene en la parada entre 15 y 90 segundos. En la fase 6 `selectParking` había mandado 3 de 22 carros al parqueadero (14 %, cerca del 10 % configurado con tan pocos datos), y se habían generado 4 buses, 3 de los cuales ya habían pasado por la parada.

El mapa de densidad de la fase 6 muestra el efecto de los semáforos. Los tramos en rojo aparecen en los accesos del norte y del este justo antes de la intersección, que es donde se acumulan los carros mientras su semáforo está en rojo; el acceso del sur se ve naranja. En el acceso del norte hay además un tramo rojo más arriba, a la altura de la parada de bus, porque cada bus detenido frena a los carros que vienen detrás. Lejos de esos puntos las vías vuelven a verde. Con fases de 30 y 20 segundos ninguna fila llega al borde del plano, así que el semáforo alcanza a evacuar los carros en cada ciclo. Como propone el tutorial, el modelo sirve para probar otras duraciones de las fases y comparar el tamaño de las filas.

8.4. Job shop (modelado de procesos)

Modela un taller que recibe estibas de materia prima, las almacena y las procesa en máquinas CNC. Viene del libro *AnyLogic in Three Days* y se construyó en cinco fases:

1. **Modelo simple.** Red de nodos y rutas sobre el plano, una estantería (*Pallet Rack*) de dos niveles y un diagrama que genera una estiba cada 5 minutos, la almacena y la retira tras `triangular(15, 20, 30)` minutos.
2. **Recursos.** Un `ResourcePool` de 5 montacargas a 1 m/s que mueven las estibas.
3. **Animación 3D** con cámara, paredes y figuras 3D de estibas y montacargas.
4. **Camiones.** Un camión llega cada hora a 40 km/h y, al descargar, llama a `sourcePallets.inject(60)`, que genera 60 estibas por camión.
5. **Máquinas CNC.** Dos máquinas definidas por su ubicación en el plano (*By home location*) que procesan las estibas durante 1 minuto.

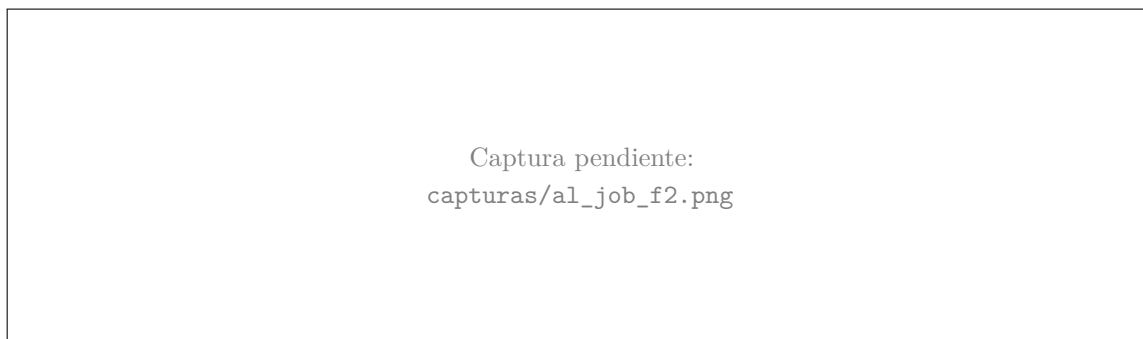


Figura 23: Job shop, fases 1 y 2: plano, estantería, montacargas y diagrama de flujo.

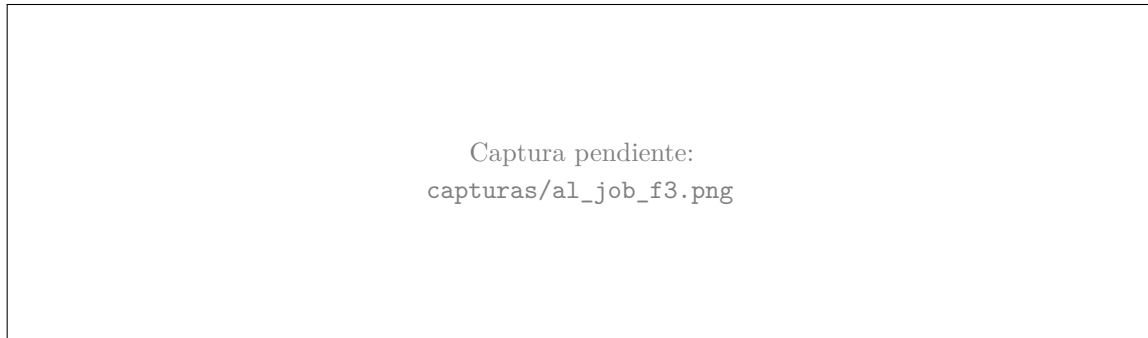


Figura 24: Job shop, fase 3: animación 3D.

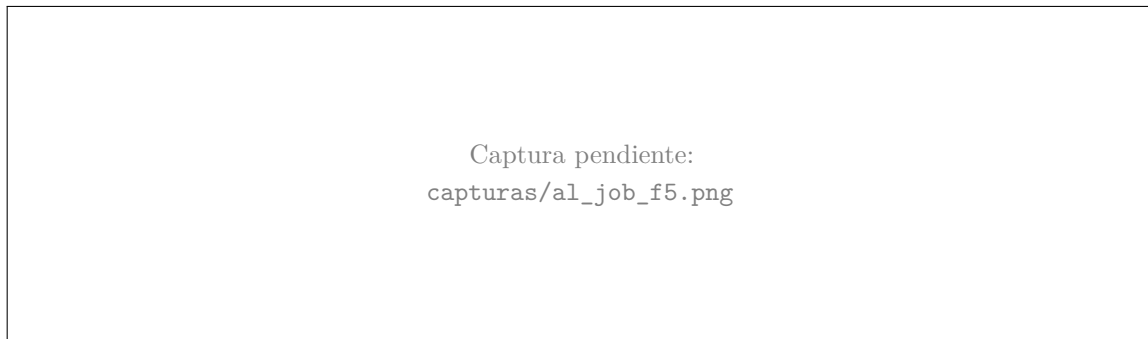


Figura 25: Job shop, fases 4 y 5: modelo completo con camiones y máquinas CNC.

8.5. Lead acid battery production (manejo de materiales)

Es el tutorial más largo. Modela una línea de producción de baterías de plomo-ácido con la *Material Handling Library*, en ocho fases: bandas transportadoras y estación de empastado de electrodos, montacargas en espacio libre que llevan los lotes al horno de curado, envoltura y ensamble de grupos de electrodos, ensamble de la batería con grúa, tapado, control de calidad y llenado de electrolito, entrega en AGV a la zona de carga, un bloque personalizado para separar ánodos y cátodos, y la producción de electrodos a partir de rollos de plomo con la *Fluid Library*.

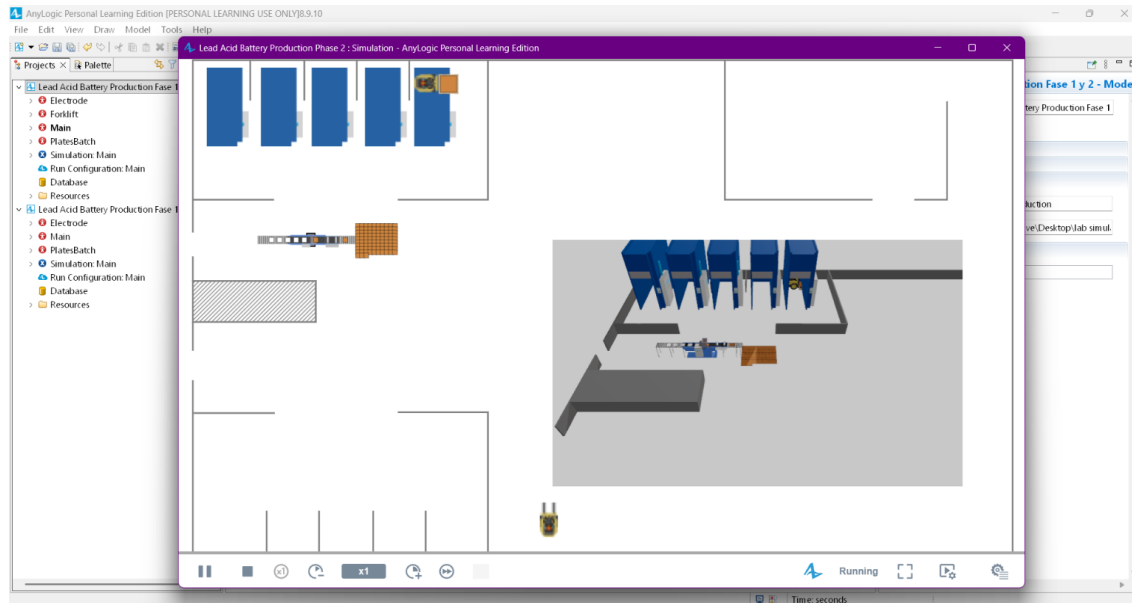


Figura 26: Lead acid battery, fases 1 y 2: banda de empastado, lotes de electrodos y montacargas que los llevan a las cámaras de curado.

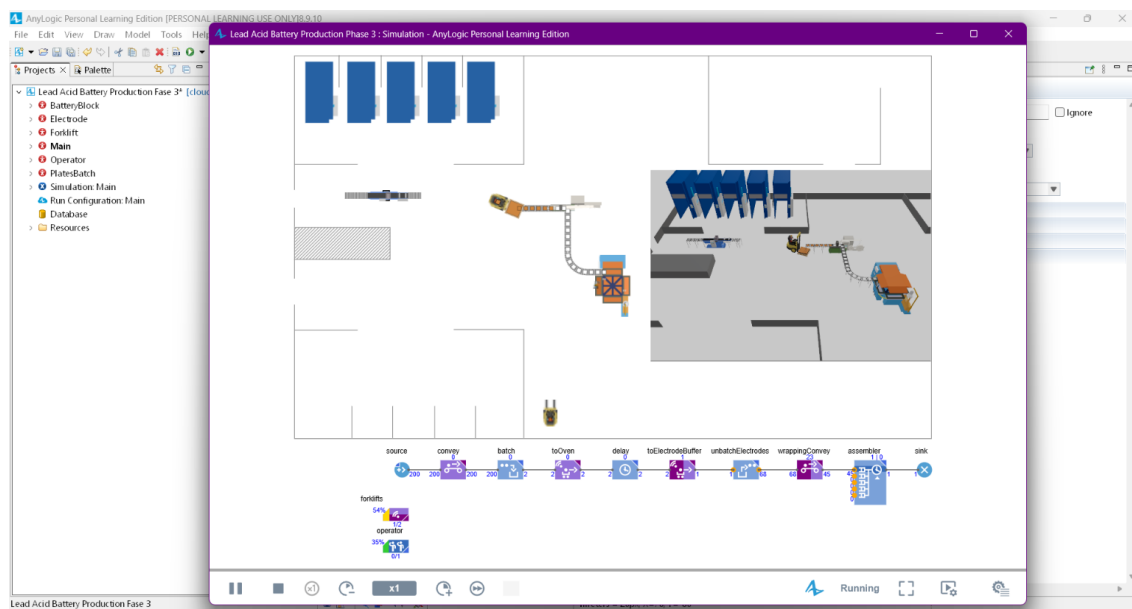


Figura 27: Lead acid battery, fase 3: envoltura de electrodos y ensamble de grupos.

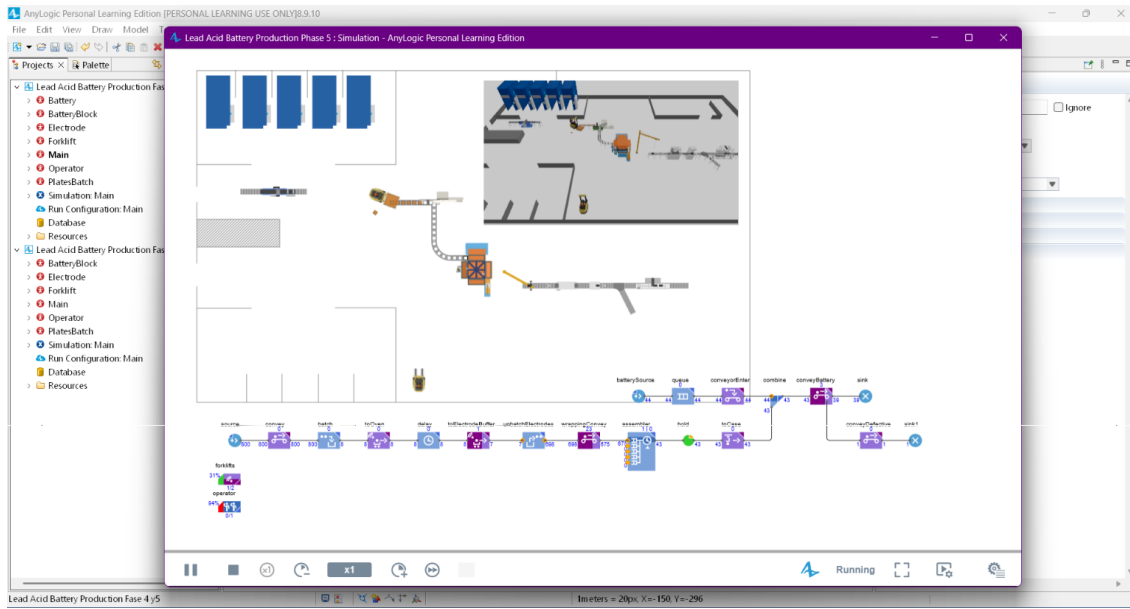


Figura 28: Lead acid battery, fases 4 y 5: ensamble de la batería, control de calidad y llenado.

Las capturas permiten seguir el flujo de material. En la fase 3 la fuente había generado 200 electrodos, que se agruparon en 2 lotes, pasaron por el horno, se desagruparon y entraron a la banda de envoltura; el ensamblador toma 15 electrodos por bloque, así que con 45 electrodos envueltos alcanzaba para 3 bloques y llevaba 1 terminado. En la fase 5, con 800 electrodos generados, el ensamblador había recibido 675 y producido 43 bloques, y de las 43 baterías que salieron de la línea, 39 pasaron el control de calidad y 1 se desvió a la banda de defectuosas (`conveyDefective`).

Lo más interesante es la utilización de los recursos. En la fase 3 los montacargas estaban ocupados el 54 % del tiempo y el operario del ensamble el 35 %. En la fase 5 los montacargas bajaron al 31 % y el operario subió al 94 %: con la línea completa, el ensamble de bloques se vuelve el cuello de botella, mientras que los montacargas tienen capacidad de sobra. Si se quisiera producir más baterías, el primer cambio sería un segundo operario en el ensamble, no más montacargas.

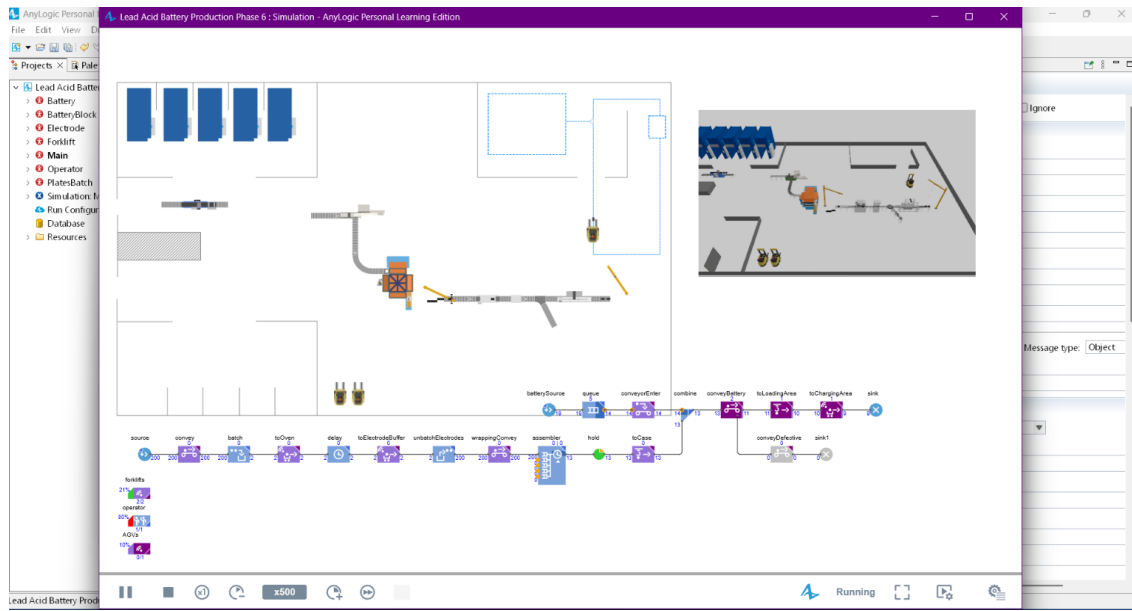


Figura 29: Lead acid battery, fase 6: entrega de las baterías a la zona de carga con AGV.

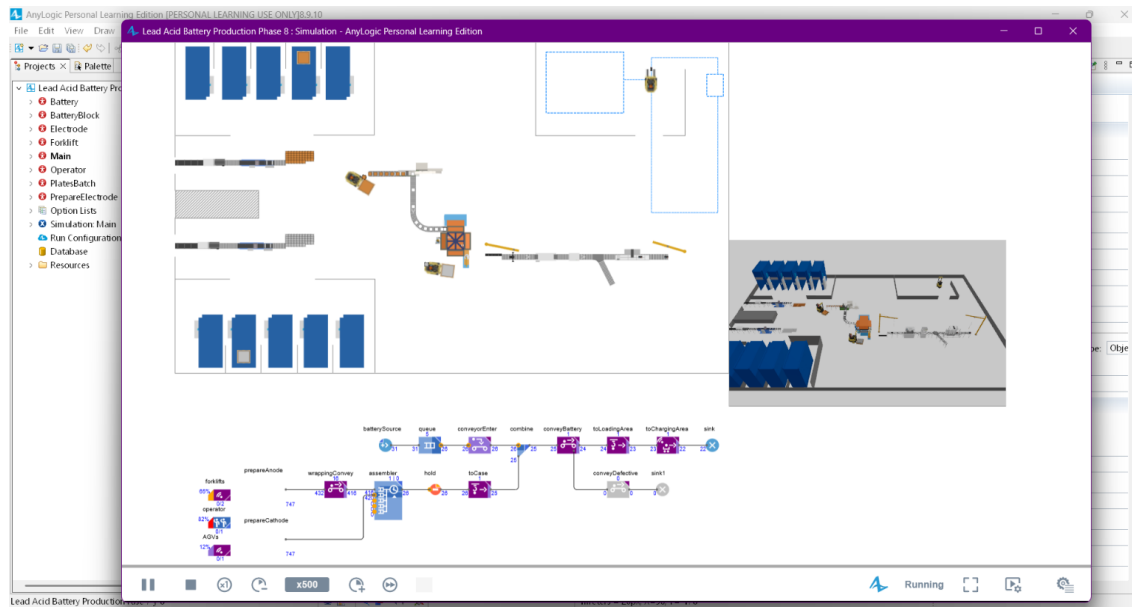


Figura 30: Lead acid battery, fases 7 y 8: ánodos y cátodos preparados por separado con el bloque personalizado PrepareElectrode y modelo completo en 3D.

En la fase 6 las baterías ya no terminan al salir de la línea: una grúa las pone en la zona de carga (`toLoadingArea`) y un AGV que sigue una ruta guiada las lleva a la zona de carga eléctrica (`toChargingArea`). En la captura habían salido 13 baterías de la línea y 9 ya estaban en la zona de carga. El AGV estuvo ocupado solo el 10 % del tiempo, así que un único vehículo alcanza de sobra para el ritmo de la línea.

Las fases 7 y 8 cambian cómo se preparan los electrodos. En lugar de una sola cadena de bloques, el tutorial encapsula las siete etapas iniciales (empastado, lotes, horno, desagrupado) en un bloque personalizado, `PrepareElectrode`, y lo usa dos veces: `prepareAnode` y `prepareCathode`, cada uno

con sus propios parámetros. En la fase 8 la materia prima de esos bloques ya no se genera como agentes sueltos, sino que sale de rollos de plomo modelados con la *Fluid Library*. En la captura cada bloque había entregado 747 electrodos, y el ensamblador, que ahora combina ánodos y cátodos, había armado 28 bloques.

Al comparar las fases 6 y 8 se ve que la carga de los recursos cambió de sitio. Los montacargas pasaron del 21 % al 66 % de ocupación, porque ahora mueven lotes de ánodos y de cátodos hacia dos zonas de curado, mientras que el operario se mantuvo alto (80 % y 82 %). En las dos capturas hay 5 carcasas esperando en la cola antes de **combine**: las carcasas llegan a 10 por hora y la línea de ensamble no alcanza a producir bloques a ese ritmo, lo que confirma que el ensamble sigue siendo el cuello de botella del modelo completo.

9. Conclusiones

1. La intensidad de tráfico explica casi todo lo observado en las colas. En los dos escenarios de la cola G/G/1, $\rho > 1$, y un cálculo de flujo tan simple como $(\lambda - \mu)T$ anticipó el número máximo de clientes (80 contra 77,6 en A y 20 contra 22,7 en B) y el tiempo para vaciar el sistema. El escenario B, con cinco veces menos clientes, fue peor para cada cliente (24,0 contra 13,0 de tiempo medio en el sistema) porque su exceso de demanda sobre la capacidad es mayor.
2. En la red de colas el cuello de botella cambió de nodo entre escenarios: en A fue la nave principal, saturada incluso a ritmo rápido ($\rho_3 = 1,43$), y en B la ventanilla ($\rho_1 = 1$). La regla de apurarse con 5 o más vehículos convierte a N3 en un regulador en el escenario B, que se mantiene cerca de 4 vehículos con la menor dispersión de todas las medidas.
3. Los intervalos de confianza mostraron que 20 repeticiones separan con claridad 9 de las 10 medidas entre los dos escenarios, pero no el número medio en la ventanilla, que es la medida más variable por trabajar con $\rho_1 \approx 1$. Para bajar el error del tiempo en el sistema al 5 % se necesitarían unas 107 corridas.
4. Validar los resultados contra la teoría sirvió para encontrar errores reales. La ley de Little reveló que el ejemplo de la red de colas subestimaba el número medio de vehículos por nodo (en N3, 16,94 en lugar de 23,15), y la revisión de índices mostró que el ejemplo G/G/1 dejaba por fuera al último cliente. En el libro también hubo que corregir tres detalles del pseudocódigo del inventario.
5. En el inventario, la política ($p = 20, P = 100$) con $L = 5$ funciona mientras la demanda esperada en el plazo de entrega, 15λ , no supere el punto de reorden. Por encima de $\lambda \approx 1,3$ los faltantes crecen rápido, y con $\lambda = 3$ uno de cada cuatro clientes no queda satisfecho. El beneficio de una sola corrida puede engañar por el pedido que llega después del cierre.
6. SimPy y AnyLogic resuelven los mismos problemas con otro enfoque: en lugar de programar la lista de sucesos, se describe lo que hace cada entidad (SimPy) o se arma el proceso con bloques gráficos (AnyLogic). Ahorran mucho código, pero no eximen de entender el mecanismo; el caso del moderador en SimPy mostró que un empate entre dos eventos simultáneos puede romper un modelo que parece correcto.

10. Anexos

Se adjuntan:

- Los cuatro cuadernos de Jupyter ejecutados de principio a fin, con los datos de todas las simulaciones y repeticiones de cada escenario y modelo: `Lab4_5.1_GG1_160005002.ipynb`, `Lab4_5.2_RedColas_160005002.ipynb`, `Lab4_5.3_Inventario_160005002.ipynb` y `Lab4_5.4_SimPy_160005002.ipynb`.
- La carpeta `Modelos_AnyLogic` con los modelos de los cinco tutoriales del numeral 5.5.

Todas las tablas y figuras de las secciones 4 a 6 salen de esos cuadernos.

Referencias

- [1] Ríos, D., Ríos, S. y Martín, J. *Simulación: métodos y aplicaciones*, 2.^a edición. Alfaomega, 2008. Capítulo 5.
- [2] Cruz Roa, A. A. 6. *Simulación de eventos discretos*. Diapositivas del curso Simulación Computacional, Universidad de los Llanos.
- [3] Scherfke, S. *Discrete-event simulation with SimPy*. EuroPython 2014. <https://stefan.sofa-rockers.org/downloads/simpy-ep14.pdf>
- [4] Team SimPy. *SimPy: Discrete event simulation for Python*, versión 4.1. Documentación y ejemplos. <https://simpy.readthedocs.io>
- [5] The AnyLogic Company. *AnyLogic Help: Tutorials* (Bank office, Subway entrance, Intersection, Job shop, Lead acid battery production). <https://anylogic.help/tutorials/>